

IUNO – Nationales Referenzprojekt IT-Sicherheit in Industrie 4.0

Deliverable

Arbeitspaket:	Demonstrator „Visueller Security Leitstand“
Unterarbeitspaket:	Implementierung von Datenquellen und nötiger Infrastruktur für den visuellen IT-Sicherheits-Leitstand
Meilenstein:	M24

D4.2-1 / M24

Anomalieerkennung im Wertschöpfungsnetzwerk; Datenaustauschplattform für das Wertschöpfungs- netzwerk

Datum:	30.06.2017
Version:	1.0
Editor:	Ricardo Hormann, Stephan Teuber (Volkswagen AG)

Autoren:	Ricardo Hormann (Volkswagen AG), Stephan Teuber (Volkswagen AG), Sebastian Nikelski (Volkswagen AG), Pedro Larbig (Fraunhofer SIT), Sebastian Neff (Fraunhofer SIT), Dr. Dirk Scheuermann (Fraunhofer SIT), Alexander Giehl (Fraunhofer AISEC), Sven Plaga (Fraunhofer AISEC), Mohammad Reza Norouzian (TUM), Alexander Malkis (TUM)
----------	--

Ansprechpartner:	Ricardo Hormann, Stephan Teuber (Volkswagen AG) Brieffach 011/1803 38436 Wolfsburg iuno@volkswagen.de
------------------	--

Dokumenteninformation

VERSIONIERUNG & DOKUMENTEN HISTORIE			
DATUM (dd.mm.yyyy)	VERSION	AUTOR	KOMMENTARE
30.06.2017	1.0	s. Autorenliste	Initiale Version

Dieses Deliverable wurde von folgenden Partnern erstellt:

- o DFKI GmbH
- o Fraunhofer AISEC
- o Fraunhofer SIT
- o TU München
- o Volkswagen AG

Inhalt

1	Anomalieerkennung - Betrachtungen zum State of the Art.....	5
1.1	Grundbegriff und Ausprägungen	5
1.2	Grundlagen der Anomalieerkennung	5
1.3	Allgemeiner Stand der Technik	5
1.4	Referenzprojekte	6
1.5	Kommerzielle Produkte	7
1.5.1	Einleitung	7
1.5.2	Produktübersicht.....	8
1.5.3	Diskussion	18
1.6	Herausforderungen für Industrie 4.0	19
1.7	Priorisierung der Produkte.....	20
1.7.1	Kriterien zur Priorisierung	20
1.7.2	Auswahl an Produkten.....	21
1.8	Kontaktaufnahme zu Herstellern	21
1.8.1	Vorgehen	21
1.8.2	Auswertung.....	23
2	Entwicklung und Integration von Methoden zur netzwerkseitigen Anomaliedetektion.....	24
2.1	TUM Anomalieerkennungswerkzeug	24
2.2	Datenextraktion und Parsing (Fraunhofer SIT)	27
2.2.1	Untersuchte Plattformen.....	28
2.2.1.1	Fraunhofer SIT Demonstrator	28
2.2.1.2	AP4 Demonstrator	29
2.2.2	Datenextraktion durch Taps	30
2.2.3	Datenextraktion mit Software Defined Networking (SDN).....	31
2.2.3.1	Kurzbeschreibung Software Defined Networking.....	31
2.2.3.2	OpenFlow 1.0.....	32
2.2.3.3	Genutzte Hardware und Software.....	34
2.2.3.4	Beschreibung der Implementierung	34
2.2.3.5	Ergebnis.....	35
2.2.4	Parsing durch Paketbeschreibungssprachen.....	35
2.2.4.1	Übersicht über existierende Forschungsprojekte	36
2.2.4.2	Funktionsweise von P4.....	37
2.2.5	Einsatz von P4 zum Parsing von Feldbusprotokollen	38
2.2.5.1	Unterstützung für Schleifen.....	38
2.2.5.2	Referenzen auf vorherige Header und Metadaten	38
2.2.5.3	TCP-Unterstützung	39
2.2.5.4	Zeitstempel	39
2.2.6	Beispielausgaben	39
2.2.6.1	Verfügbare Implementierungen.....	42
2.3	Interpretation der extrahierten Daten und Fehlererkennung durch Simulation (Fraunhofer SIT)	46
2.3.1	Interpretation von Feldbusprotokollen	46
2.3.1.1	Interpretation von Modbus-Nachrichten aus OpenPLC-Steuerungen	46
2.3.1.2	Interpretation von Profinet-Datenpaketen in Simatic-Installationen.....	48
2.3.2	Implementierung von Interpretern	49
2.3.2.1	Grammatikdefinition	50
2.3.2.2	Parsererzeugung	50
2.3.2.3	Beispielgrammatik.....	50
2.3.3	Extraktionsmethoden	51
2.3.4	Fehlererkennungsmethoden – Die Simulation	51
2.3.5	Fehlererkennungsmethoden – Der Erkenner	53
2.3.5.1	Sollwertvergleiche.....	53
2.3.5.2	Zustandsübergangsüberwachung.....	54
2.3.5.3	Beispielausgaben.....	55
2.3.6	Gesamtarchitektur	56
2.3.7	Perspektiven	57
2.4	SpotFactory und N-Grams Anomaly	59

2.5	Verteiltes DFKI-Anomalieerkennungswerkzeug	60
2.6	Strategien für Anomalieerkennung in Legacy Feldbus-Netzwerken	61
3	Aufbau der Datenaustauschplattform (VW).....	63

1 Anomalieerkennung - Betrachtungen zum State of the Art

1.1 Grundbegriff und Ausprägungen

Anomalie bedeutet abweichendes Systemverhalten oder abweichende Systemzustände von der beabsichtigten Verhaltensweise oder dem gewünschten Zustand. Die wesentliche Aufgabe der Anomalieerkennung ist die Erkennung der Art des abweichenden Verhaltens. Manche Anomalien sind trivial und „direkt messbar“, z.B. abweichende Größen von produzierten Werkstücken oder zu langsamer Betrieb einer Maschine. Die zu messenden Zustandsmengen werden hier als „Signaturen“ bezeichnet.

Die eigentlichen „komplexen Anomalien“ äußern sich nicht direkt durch „unerlaubte Zustände“ sondern nur durch ein „auffälliges Systemverhalten“. Beispiele hierfür sind ungewöhnlich häufige Zugriffsversuche auf eine Datenbank oder ungewöhnlich häufige Zurückweisungen eines Benutzers wg. falscher Kennwort-Eingabe; ein solches Systemverhalten deutet auf unberechtigte Zugriffsversuche hin.

Gemäß obiger Beschreibung unterscheidet man in der Regel ausdrücklich zwischen den Begriffen Anomalie und Signatur, d.h. mit dem Begriff der Anomalie sind die komplexeren Sachverhalte gemeint.

1.2 Grundlagen der Anomalieerkennung

Ein wichtiger grundlegender Schritt bei der Anomalieerkennung ist die Festlegung von Soll- und Istwerten für die zu analysierenden Größen. Dabei müssen entsprechende Toleranzen für die Abweichung anwendungsspezifisch festgelegt werden. Bei direkt messbaren, trivialen Anomalien ergeben sich die Soll- und Istwert (für die „Signaturen“) direkt aus der Anwendung. Bei komplexeren, nicht direkt messbaren Anomalien ist dagegen eine Korrelation mehrerer Messdaten notwendig, und Kenngrößen mit Toleranzen müssen eigens definiert werden.

1.3 Allgemeiner Stand der Technik

Anomalieerkennung wird meistens auf Netzwerk-Ebene betrachtet [1] [2]. Bei der Analyse erfolgt schwerpunktmäßig eine Kontrolle der übers Netzwerk übertragenen Daten. In [1] findet man am Beispiel der Netzwerk-basierten Anomalieerkennung auch eine ausführliche Erläuterung grundlegender Strategien sowie eine Auflistung akademischer Entwicklungen von Tools.

Eine wichtige Rolle bei intelligenten Systemen zur Anomalieerkennung spielen auch statistische Analysen von Datenströmen. Hier gab es schon einige Entwicklungen. Ein häufig betrachtetes Anwendungsbeispiel bei bisherigen akademischen Entwicklungen ist die Energieversorgung. Als Grundlage dienen verschiedene anwendungsspezifische IEC-Standards, z.B. für Power System-Management [3], Schaltanlagen [4], Unternehmensführungs- und Leitsysteme [5].

1.4 Referenzprojekte

Das Thema Anomalieerkennung spielte bisher eine Rolle bei den folgenden BMBF-Projekten unter Beteiligung von Fraunhofer SIT oder Fraunhofer AISEC. Die Aufzählung wird in alphabetischer Reihenfolge gegeben:

- **ANSII:** Anomalieerkennung in industriellen Informationssystemen (*Fraunhofer SIT*)
 - o Betrachtung von Anomalieerkennung bei Netzwerk-Datenverkehr
 - o Beispielanwendung Industrial Control Systems
 - o Teilweise Anwendung des Metadaten-Modells aus ESUKOM
 - o Feststellung: Standard-SotA-Methoden erweisen sich als nutzlos, es müssen Szenario-spezifische Anomalien beschrieben werden!
- **ECOSSIAN:** European Control System Security Incident Analysis Network (*Fraunhofer AISEC*)
 - o Anwendung von Anomalieerkennung zur Angriffserkennung auf kritische Infrastrukturen
 - o Entwicklung eines europaweit vernetzten Monitoring- und Frühwarnsystems für grenzübergreifendes Katastrophenmanagement
- **ESUKOM:** Echtzeit-Sicherheit für Unternehmensnetzwerke; hier u.a. mehrteilige Betrachtung des Anomalie-Begriffs (*Fraunhofer SIT*)
 - o Anomalieerkennung als eine von mehreren „Kernanforderungen“
 - o Definition von kategorisierten Features und zugehörigen Attributen für alle Kernanforderungen – hierfür Spezifikation eines IF-MAP-basierten Metadatenmodells.
 - o Definition von Komponenten für Signaturen und für Anomalieerkennung (Features + Prozeduren) – auch für andere Kernanforderungen
 - o Entwicklung einer „Detection Engine“ zur Konsolidierung (Korrelation) von Metadaten für Anomalieerkennung
- **SIMU:** Entwicklung von Secure Information und Event Management Systemen mit Verwendbarkeit bei KMUs (Nachfolgeprojekt von ESUKOM, *Fraunhofer SIT*)

1.5 Kommerzielle Produkte

In diesem Abschnitt werden die recherchierten Produkte, die sich mit Anomalieerkennung in (industriellen) Netzen befassen, vorgestellt. Die gewählte Darstellungsweise wird in Abschnitt 5.1 näher erläutert. Die recherchierten Produkte werden in Abschnitt 5.2 genauer beschrieben. Eine Diskussion der gefundenen Produkte wird abschließend in Abschnitt 5.3 gegeben.

1.5.1 Einleitung

Die relevanten Produkte werden pro Hersteller in einer separaten Tabelle angegeben. Das Template für die Tabelle ist unterstehend gegeben.

Hersteller	Name des Herstellers			Hauptsitz	Ländernamen		
Produkt(e)	Name des Produkts bzw. Namen der Produkte			Domänen	Anwendungsdomänen des Produkts		
Host/Network	Host oder Netzwerk	IDS Typ	Anomaly, Signature	HW?	Ja/Nein	IPS?	Ja/Nein
Bemerkungen	Bemerkungen zum Produkt						
Geschäftsmodell	Angaben zum Geschäftsmodell des Herstellers						
Referenzen	Referenz zum Produkt						

Der Name des Herstellers sowie dessen Hauptunternehmenssitz sind jeweils in der ersten Zeile angegeben. Der Name des Produkts ist zusammen mit der Anwendungsdomäne laut Hersteller in der zweiten Zeile zu finden. Die dritte Zeile beschreibt die Eigenschaften der untersuchten Produkte. Die Zelle *Host/Network* trifft eine Unterscheidung zwischen dem Typ des betrachteten Intrusion Detection Systems (IDS) nach dessen Einsatzbereich (siehe Kapitel 3). Host-basierte IDS werden auf einem einzelnen Host ausgeführt und dienen zu dessen Überwachung, wohingegen Netzwerk-basierte IDS den Verkehr im gesamten Netzwerk auf Anomalien hin überprüfen. Der Typ des IDS (Anomalieerkennung oder signaturbasiertes Verfahren, siehe Kapitel 1) ist in der zweiten Zeile angegeben. Die dritte Zeile beschreibt, ob der Hersteller sein Produkt zusammen mit zugehöriger Hardware ausliefert oder ob es sich um ein reines Softwareprodukt handelt. Die letzte Zeile beschreibt,

ob das Produkt, zusätzlich zur passiven Erkennung (Intrusion Detection) auch Maßnahmen zur aktiven Verhinderung detektierter Vorfälle (Intrusion Prevention) anbietet. Solche Systeme werden als Intrusion Prevention System (IPS) bezeichnet [6]. Bemerkungen zum Produkt, Erklärungen zum Geschäftsmodell sowie Referenzen werden in den folgenden Zeilen gegeben.

1.5.2 Produktübersicht

Im Folgenden wird eine nach Herstellernamen sortierte Übersicht der recherchierten Produkte gegeben:

Hersteller	ANDATA			Hauptsitz	Österreich		
Produkt(e)	(keine nähere Bezeichnung)			Domänen	U.a. Überwachung von Maschinen		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	?	IPS?	Nein
Bemerkungen	Lösungen auf der Basis von Machine Learning						
Geschäftsmodell	-						
Referenzen	http://www.andata.at/de/produkte-loesungen/anomalieerkennung.html						

Hersteller	Apex Technology Services			Hauptsitz	USA		
Produkt(e)	-			Domänen	Industrial Internet of Things		
Host/Network	Netzwe rk	IDS Typ	Anomaly	HW?	Nei n	IPS?	Nein
Bemerkungen	„Unsupervised machine learning based on non-linear behavioral clustering using self-organizing maps“ - Virtuelle Maschine						
Geschäftsmodell	„Software as a Service“ (siehe www.apextechservices.com/support-packages/default.aspx#ApexEssentials)						
Referenzen	www.apextechservices.com/cybersecurity/default.aspx#anomalydetection						

Hersteller	Checkpoint			Hauptsitz	USA, Israel		
Produkt(e)	1200R Rugged Appliance			Domänen	Industrielle Steuernetze		
Host/Network	Networ k	IDS Typ	Anomaly, Signature	HW?	Ja	IPS?	Ja
Bemerkungen	- Unterstützung für BACNet, DNP3, IEC-60870-5-104, IEC 60870-6 (ICCP), IEC 61850, MMS, Modbus, OPC, Profinet, S7 (Siemens), etc. - Stateful inspection - Firewall und Antivirus Programme sind im Produkt enthalten - Die Beschreibung auf der Produktseite lässt darauf schließen, dass einerseits Anomalieerkennung und andererseits auch Signaturen verwendet werden						
Geschäftsmodell	-						
Referenzen	https://www.checkpoint.com/products/industrial-control-systems-appliances/index.html						

Hersteller	Cisco			Hauptsitz	USA		
Produkt(e)	Cisco ASA 5506H-X with FirePOWER Services Ruggedized Security Appliance			Domänen	Industrielle Steuernetze		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Ja	IPS?	Ja
Bemerkungen							
Geschäftsmodell	-						
Referenzen	https://www.cisco.com/c/en/us/products/security/asa-firepower-services/ruggedized.html						

Hersteller	Consistec			Hauptsitz	Deutschland		
Produkt(e)	Caplon (Produktserie)			Domänen	Überwachung von Geschäftsprozess en		

Host/Network	Networ k + Host	IDS Typ	Anomaly	HW?	Ja	IPS?	Nein
Bemerkungen	• Network Recorder zur Aufklärung von Netzwerkproblemen und Sicherheitsvorfällen • Caplon Automation auch für Fernbedienung (Fernwartung?) • Service Monitoring auch zur Erkennung von Server-Problemen						
Geschäftsmodell	-						
Referenzen	https://www.consistec.de/de/produkte/						

Hersteller	CyberFlow Analytics			Hauptsitz	USA		
Produkt(e)	Flowscape			Domänen			
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Nei n	IPS?	Ja
Bemerkungen	• Ausgeführt in virtuellen Maschinen und Docker Containern						
Geschäftsmodell	-						
Referenzen	http://www.cyberflowanalytics.com/site/product/						

Hersteller	CyberX			Hauptsitz	USA		
Produkt(e)	XSense			Domänen	U.A. Manufacturing		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Nei n	IPS?	Nein
Bemerkungen	• Plug'n'Play Installation • Machine Learning						
Geschäftsmodell	-						
Referenzen	http://cyberx-labs.com/xsense/						

Hersteller	DarkTrace			Hauptsitz	Großbritannien		
Produkt(e)	Industrial Immune System			Domänen	Industrielle Steuernetze		

Host/Network	Networ k	IDS Typ	Anomaly	HW?	Nei n	IPS?	Nein
Bemerkungen	- Machine Learning, Bayessche Statistik - Auflistung auf Herstellerseite was durch das Tool gefunden wird						
Geschäftsmodell	-						
Referenzen	https://www.darktrace.com/products/#industrial-immune-system						

Hersteller	gateprotect (Rohde & Schwarz)			Hauptsitz	Deutschland		
Produkt(e)	gateprotect NP (Produktreihe)			Domänen	U.A. Kritische Infrastrukturen		
Host/Network	Networ k	IDS Typ	Signatur + Anomalie	HW?	Ja	IPS?	Ja
Bemerkungen	- Erste Hersteller der rhebo Gründer, aufgekauft und umbenannt durch Rhode & Schwarz - Ursprünglich rein Office IT, mittlerweile auch Produkte im Industriekontext - Nur vollständig validierte Kommunikation wird weitergeleitet						
Geschäftsmodell	-						
Referenzen	http://www.gateprotect.de/de/Produkte/Next-Generation-Firewall						

Hersteller	Guardian Analytics			Hauptsitz	USA		
Produkt(e)	FraudMap Access			Domänen	Online-Banking		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Nei n	IPS?	Ja
Bemerkungen	Verhaltensanalyse bei Benutzer-Accounts						
Geschäftsmodell	-						
Referenzen	http://www.guardiananalytics.com/products/fraudMAP-access.php						

Hersteller	ICS ²			Hauptsitz	Israel		
Produkt(e)	On-Guard Analyzer Active Guard			Domänen	Industrielle Prozesse		

Host/Network	Networ k	IDS Typ	Anomaly	HW?	Nei n	IPS?	Nein
Bemerkungen	Fokus liegt auf Prozessüberwachung in Kraftwerken						
Geschäftsmodell	-						
Referenzen	http://ics2.com/product-solution/						

Hersteller	Indegy			Hauptsitz	Israel		
Produkt(e)	Industrial Cyber Security			Domänen	Industrielle Steuernetze		
Host/Network	?	IDS Typ	?	HW?	?	IPS?	?
Bemerkungen	Wenig Information über Hersteller und Produkt verfügbar						
Geschäftsmodell	-						
Referenzen	http://www.indegy.com/industrialcybersecurity/						

Hersteller	Infoblox			Hauptsitz	USA		
Produkt(e)	DNS Secure (Produktserie)			Domänen	Informationsdiens te + Regierung		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Ja	IPS?	Ja
Bemerkungen	- Mehrere Varianten zum Schutz von DNS - Auch Firewalls gegen Malware.						
Geschäftsmodell	-						
Referenzen	https://www.infoblox.com/products/secure-dns						

Hersteller	Juniper			Hauptsitz	USA		
Produkt(e)	LN2600 Rugged Secure Router			Domänen	Industrielle Steuernetze		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Ja	IPS?	Ja

Bemerkungen	•
Geschäftsmodell	-
Referenzen	https://www.juniper.net/assets/us/en/local/pdf/datasheets/1000423-en.pdf

Hersteller	Kyland			Hauptsitz	China		
Produkt(e)	Agate7200			Domänen	U.A. Fabrik- automatisierung		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Ja	IPS?	?
Bemerkungen	Unterstützte Protokolle: Modbus TCP, IEC61850 (GOOSE \ SV \ MMS), ETHERNET/IP, ETHERCAT, POWERLINK, PROFINET, OPC, IEC-104, DNP3						
Geschäftsmodell	-						
Referenzen	http://www.kyland.com/product/Agate7200.html						

Hersteller	Lancope (Cisco)			Hauptsitz	USA		
Produkt(e)	StealthWatch FlowCollector StealthWatch FlowSensor StealthWatch Management Console			Domänen	U.A. Manufacturing		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Ja	IPS?	Ja
Bemerkungen	- Verschiedene Hardwarevarianten - Verschiedene Produkte werden in Kombination in das Netzwerk eingebracht: <i>Flowsensors</i> sammeln Daten, <i>FlowCollector</i> aggregieren diese und eine <i>Management Console</i> bereitet diese auf						
Geschäftsmodell	-						
Referenzen	https://www.lancope.com/products/stealthwatch-flowcollector						

Hersteller	Netsecuris Inc.			Hauptsitz	USA		
Produkt(e)	Industrial Control Systems (ICS) and SCADA Security Services			Domänen	Industrielle Steuernetze		

Host/Network	?	IDS Typ	Signature	HW?	Nein	IPS?	Ja
Bemerkungen	Wenig Information verfügbar						
Geschäftsmodell	-						
Referenzen	http://www.netsecuris.com/index.php/ics-scada-security						

Hersteller	NexDefense			Hauptsitz	USA		
Produkt(e)	Sophia			Domänen	Power, Oil and Gas, Defense		
Host/Network	Network	IDS Typ	Anomaly	HW?	Nein	IPS?	Nein
Bemerkungen	- Kollaboration aus United States Department of Energy, Battelle Energy Alliance und Idaho National Laboratory - Einbringung der Komponenten auf der Prozessleitebene und Steuerungsebene						
Geschäftsmodell	-						
Referenzen	http://www.nexdefense.com/products-and-services/sophia/ [7]						

Hersteller	Prelert			Hauptsitz	USA		
Produkt(e)	Anomaly Detective			Domänen	U.a. Einzelhandel		
Host/Network	Network	IDS Typ	Anomaly	HW?	Nein	IPS?	Nein
Bemerkungen	Automatisches Anlernen ohne vorheriges manuelles Training						
Geschäftsmodell	-						
Referenzen	http://info.prelert.com/products/anomaly-detective						

Hersteller	ProAct			Hauptsitz	Großbritannien		
Produkt(e)	ISS Real Secure			Domänen	Informationsdienste		

Host/Network	Networ k + Host	IDS Typ	Anomaly Signature	+	HW?	Nei n	IPS?	Ja
Bemerkungen	- Verschiedene Software-Produkte auf verschiedenen Ebenen (Server, Network) - Wurde 2003 im Rahmen einer Diplomarbeit mit Open Source verglichen! [8]							
Geschäftsmodell	-							
Referenzen	http://www.tech.proact.co.uk/iss/realsecure_protection_software.html							

Hersteller	radiflow				Hauptsitz	USA, Israel		
Produkt(e)	iSID				Domänen	Industrielle Steuernetze		
Host/Network	Networ k	IDS Typ	Anomaly		HW?	Nei n	IPS?	Nein
Bemerkungen	Lösung besteht aus einer zentralen „Radiflow IDS“ und mehreren „Radiflow Secure Gateways“, die an verschiedenen Standorten angebracht werden							
Geschäftsmodell	-							
Referenzen	http://radiflow.com/products/isid-intrusion-detection-system/ [9]							

Hersteller	rhebo				Hauptsitz	Deutschland		
Produkt(e)	Industrial Protector				Domänen	Industrielle Steuernetze		
Host/Network	Networ k	IDS Typ	Anomaly		HW?	Ja	IPS?	Nein
Bemerkungen	- Deep Packet Inspection - Anomalieerkennung über automatisch gelerntes Kommunikationsmuster - Plug'n'Play Komponente - Grafische Visualisierung der Netzwerkkommunikation							
Geschäftsmodell	-							
Referenzen	http://rhebo.com/de/#protector							

Hersteller	SCADAfence			Hauptsitz	Israel		
Produkt(e)	-			Domänen	U.A. Manufacturing		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Nei n	IPS?	Nein
Bemerkungen	Auf der Herstellerseite sind nur wenige Informationen zum Produkt zu finden						
Geschäftsmodell	-						
Referenzen	http://www.scadafence.com/						

Hersteller	Security Matters			Hauptsitz	Niederlande		
Produkt(e)	SilentDefense ICS			Domänen	U.A. Manufacturing		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Nei n	IPS?	Nein
Bemerkungen	- Deep Protocol Behavior Inspection - Modulbasierte Unterstützung für Protokolle, z.B., Modbus, MMS, OPC, SMB - Einbringung der „Command Center“-Komponenten auf der Betriebsebene und der „Monitoring Sensor“-Komponenten auf den darunterliegenden Ebenen						
Geschäftsmodell	-						
Referenzen	http://www.secmatters.com/products-ics [10]						

Hersteller	Sentryo			Hauptsitz	Frankreich		
Produkt(e)	ICS Cybervision			Domänen	Manufacturing		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Nei n	IPS?	Nein
Bemerkungen							
Geschäftsmodell	-						

Referenzen	https://www.sentryo.net/
-------------------	---

Hersteller	Siga Cyber Alert Systems			Hauptsitz	Israel		
Produkt(e)	-			Domänen	U.A. Manufacturing		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Ja	IPS?	Nein
Bemerkungen	- Machine Learning - Einbringung der Komponenten auf der Feldebene: Hardwarekomponente zwischen PLC und dem jeweiligen industriellen Endgerät - proprietäre Serversoftware						
Geschäftsmodell	-						
Referenzen	http://www.sigasec.com/						

Hersteller	ThetaRay			Hauptsitz	Israel		
Produkt(e)	-			Domänen	Manufacturing		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Nei n	IPS?	Nein
Bemerkungen	- Virtuelle Maschine - Auch als reine Software-Lösung verfügbar						
Geschäftsmodell	-						
Referenzen	http://www.thetaray.com/critical-manufacturing/						

Hersteller	Tofino Security (Belden)			Hauptsitz	Kanada		
Produkt(e)	Tofino Xenon Security Appliance Loadable Security Modules (LSMs)			Domänen	Industrielle Steuernetze		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Ja	IPS?	Ja
Bemerkungen	- Über LSMs IDPS Unterstützung für Modbus, OPC, EtherNet/IP über Deep Packet Inspection möglich - Lösung zu Schaffung von „Security-Zonen“ - Einbringung der Komponenten auf der Unternehmungsebene bis						

	hinunter zur Feldebene
Geschäftsmodell	-
Referenzen	https://www.tofinosecurity.com/products/tofino-security-appliance [11]

Hersteller	Videc			Hauptsitz	Deutschland		
Produkt(e)	Industrie Risiko Management Automatisierung (IRMA)			Domänen	Industrielle Steuernetze		
Host/Network	Networ k	IDS Typ	Anomaly	HW?	Ja	IPS?	Nein
Bemerkungen	• Passive Überwachung von IT-Systemen und Produktionsanlagen • Incident Reporting bei Sicherheitsvorfällen • Linuxsystem mit Open Source Komponenten • Anomalieerkennung als optionales Zusatzmodul verfügbar • Einbringung mindestens auf Höhe der Betriebsleitebene bis hinunter zur Feldebene						
Geschäftsmodell	-						
Referenzen	http://www.videc.info/de/produkte/irma						

1.5.3 Diskussion

In diesem Abschnitt werden die recherchierten Produkte aus Abschnitt 5.2 näher diskutiert. Insgesamt wurden 29 Produkte recherchiert, die in die folgenden Betrachtungen einfließen. Es werden Gemeinsamkeiten und Auffälligkeiten näher dargestellt und zusammengefasst.

Viele der untersuchten Hersteller bieten einen „*analyst-in-the-box*“ an, wie es auch von [6] beobachtet wurde. Diese Produkte zeichnen sich typischerweise dadurch aus, dass ein Hardware-Komponente angeboten wird, die durch *plug-n-play*-ähnliche Funktionalität an das bestehende, industrielle Netz angeschlossen werden. Diese überwachen den Verkehr im Netzwerk und überprüfen ihn auf Anomalien. Die Produkte benötigen eine Trainingsphase, in der der Netzverkehr aufgezeichnet und dann als zukünftiger Sollwert (siehe Kapitel 2) betrachtet wird.

Bei den Lösungen einiger Hersteller ist eine Kombination unterschiedlicher Komponenten angedacht. Üblicherweise handelt es sich hierbei um eine zentrale Komponente zur Aggregation und Auswertung der Daten, die durch mehrere Sensorkomponenten gesammelt

werden. Diese werden an unterschiedlichen Ansatzpunkten in das industrielle Steuernetzwerk eingebracht.

Einige der betrachteten Hersteller geben weiter an, dass durch ihr Produkt die Zahl der *False Positives*, also jener Fehlermeldungen, denen kein tatsächlich stattfindender Angriff zugrunde liegt, reduziert wird. *False Positives* müssen durch den Einsatz eines Menschen näher untersucht werden, was einen kostenintensiven Faktor in der Netzwerküberwachung darstellt [6].

Die regionale Verteilung der Hersteller konzentriert sich stark auf bestimmte Regionen. Der Großteil der Hersteller hat seinen Firmensitz in Nordamerika, es gibt nur wenige Hersteller aus der EU. Diese verteilen sich auf Deutschland, Österreich, Niederlande, Großbritannien und Frankreich. Eine Auffälligkeit ist die hohe Anzahl von Herstellern aus Israel, bei denen es sich oft um Start-Ups handelt. Diese Auffälligkeit kann durch staatliche Förderprogramme erklärt werden [12] [13] [14].

Angaben zum Geschäftsmodell der Hersteller oder zu den Preisen sind auf den Internetpräsenzen der jeweiligen Hersteller nicht oder nur sehr spärlich zu finden. Eine direkte Kontaktaufnahme mit den Herstellern könnte hier zu näheren Informationen führen.

Im Fall der Produkte ist auch genau auf die Wortwahl der Hersteller hinsichtlich Intrusion Prevention zu achten. Wenn ein Produkt bei Intrusion Prevention „hilft“ oder Intrusion Prevention „unterstützt“, kann dies ein Hinweis darauf sein, dass die eigentliche Verhinderung der Angriffe an anderer Stelle durchgeführt werden muss und das Produkt hierbei nicht aktiv unterstützt.

1.6 Herausforderungen für Industrie 4.0

In diesem Kapitel erfolgt ein Abgleich mit dem Stand der Technik in aktuellen Fertigungsanlagen (die sogenannte Industrie 3.0) mit den Herausforderungen, die sich aus einer Vernetzung von Wertschöpfungsketten (Industrie 4.0) ergeben.

Anomaliezustände müssen bisher weitgehend statisch von Hand definiert werden. Deshalb besteht die große Herausforderung bei Industrie 4.0 in der selbständigen, automatischen Definition von Anomaliezuständen bei Produktionsmaschinen. Weiterhin sind bisherige Anomalieerkennungssysteme zwar intelligent, geben aber als Ergebnis nur Alarm oder stoppen im Notfall die Anlage. Die Beseitigung muss dagegen hinterher manuell erfolgen. In Industrie 4.0 müssen Anomalien automatisch korrigiert werden so dass der Anlagenbetrieb weiterläuft. Aus diesem Grund ist die bereits vorher erwähnte Unterstützung der IPS-Funktionalität (neben der reinen IDS-Funktionalität) von entscheidender Bedeutung.

1.7 Priorisierung der Produkte

In diesem Kapitel wird eine Priorisierung der in Kapitel 5 diskutierten Produkte durchgeführt. Hierzu werden die Kriterien, die zur Priorisierung herangezogen werden, in Abschnitt 6.1 vorgestellt. Die Priorisierung anhand dieser Kriterien erfolgt in Abschnitt 6.2.

1.7.1 Kriterien zur Priorisierung

Host vs. Netzwerk: Sowohl auf Seite der wissenschaftlichen Betrachtungen als auch bei den verfügbaren kommerziellen Produkten konzentriert sich die Anomalieerkennung auf die Netzwerk-Ebene. Nur sehr wenige Produkte bieten außerdem eine Host-basierte Anomalieerkennung an. Im I4.0-Umfeld geht es insbesondere um vernetzte Produktion sowie um Fernwartung von Anlagen über Netzwerke. Unter diesem Aspekt erscheint es sinnvoll, bei Netzwerk-basierten Lösungen zu bleiben, d.h. die Unterstützung zusätzlicher Host-basierter Anomalieerkennung ist kein besonderes Auswahlkriterium.

Anomalie vs. Signaturen: Die wissenschaftliche Betrachtung von Anomalieerkennung konzentriert sich vorwiegend auf echte, komplexe Anomalien, und auch die verfügbaren kommerziellen Produkte unterstützen zum größten Teil keine zusätzliche Erkennung von Signaturen. Gerade bei professionellen Angreifern ist davon auszugehen, dass sie ihre Angriffe so tarnen, dass man sie mit Hilfe von Signaturen nur schwer erfassen kann (einfach zu messende Werte wie z.B. Orts- und Zeitinformationen sind leicht zu fälschen). Somit kann man sich auch bei der Priorisierung verfügbarer Lösungen auf die komplexe Anomalieerkennung konzentrieren, und die zusätzliche Unterstützung von Signaturerkennung ist kein wesentliches Kriterium.

Hardware vs. Software-Lösungen: Die Verwendung rein proprietärer Software ist für den vorgesehenen Demonstrator ungünstig, da man hier kaum Möglichkeiten zur Weiterentwicklung hat. Besser ist es, hier mit OpenSource-Software zu arbeiten. Diese Möglichkeit ist bei reinen Software-Lösungen nicht gegeben, während man vorhandene Hardware auch auf entsprechende OpenSource-Software anpassen kann. Von daher sind unter den kommerziellen Lösungen solche in die engere Auswahl zu ziehen welche auch mitgelieferte Hardware beinhalten.

IPS-Unterstützung: Wie vorher schon erwähnt, sind die automatische Beseitigung von Anomalien und die aktive Abwehr von Angriffen wichtige Herausforderungen für Industrie 4.0. Deshalb ist die Unterstützung der IPS-Funktionalität ein entscheidendes Kriterium für die Priorisierung vorhandener Lösungen.

Gesamt-Fazit: Unter den in Abschnitt 5.2 aufgeführten Produkten sind diejenigen mit zugehöriger Hardware und IPS-Unterstützung näher zu betrachten. Ebenso ist bei nicht-kommerziellen Lösungen auf diese Punkte zu achten.

1.7.2 Auswahl an Produkten

Basierend auf den Kriterien, die in Abschnitt 6.2 definiert und beschrieben sind, kann eine Auswahl für die betrachteten Produkte (siehe Abschnitt 5.2) getroffen werden. Es werden jene Produkte für eine nähere Betrachtung ausgewählt, bei denen die Kategorien *HW* und *IPS* jeweils mit *Ja* beantwortet werden kann. Dadurch verkürzt sich die Produktliste wie in folgender, weiterhin alphabetisch sortierter, Aufzählung angegeben:

- Checkpoint: 1200R Rugged Appliance
- Cisco: ASA 5506H-X with FirePOWER Services Ruggedized Security Appliance
- gateprotect (Rohde & Schwarz): gateprotect NP
- Infoblox: DNS Secure (Produktserie)
- Juniper: LN2600 Rugged Secure Router
- Lancope (Cisco): StealthWatch FlowCollector
- Tofino Security (Belden): Tofino Xenon Security Appliance mit den Loadable Security Modules (LSMs)

1.8 Kontaktaufnahme zu Herstellern

In diesem Kapitel werden die Ergebnisse der Produktpriorisierung (siehe Kapitel 6) anhand der geführten Herstellerinterviews diskutiert. Dazu wird zuerst das Vorgehen bei der Kontaktierung der Hersteller in Abschnitt 8.1. detailliert. Die Ergebnisse der Herstellerinterviews werden anschließend in Abschnitt 8.2 diskutiert.

1.8.1 Vorgehen

Zur weiteren Evaluierung der in Abschnitt 6.2 priorisierten Produkte wurde der Kontakt zu den Herstellern gesucht. Als Grundlage für die Kontaktaufnahme diente ein Fragebogen, der die Ergebnisse der zuvor durchgeführten Marktanalyse berücksichtigt. Die formulierten Fragen lauten wie folgt:

- Wie grenzt sich das Produkt von bestehenden, quelloffenen Lösungen wie bspw. BRO oder Suricata ab?
 - Welche Algorithmen werden zur Anomalieerkennung eingesetzt? Handelt es sich um Eigenentwicklungen oder werden publizierte Ansätze weiterverfolgt?
- Welches Betriebssystem kommt auf dem Gerät zum Einsatz?

- Können eigens entwickelte Softwaremodule auf dem Gerät installiert werden?
 - o Unter welcher Lizenz wird die Software vertrieben?
 - o Kann die bestehende Software ggf. gänzlich durch eine alternative Software ausgetauscht werden?
- Wie sieht das Geschäftsmodell im Detail aus?
 - o Sind Service- bzw. Wartungsverträge beim Kauf des Produkts notwendig?
 - o Wieviel kostet das Produkt?
- In welchem Umfang sind Schulungen zum Einsatz und zur täglichen Verwendung des Produkts notwendig?
 - o In welcher Form werden Schulungen angeboten (Präsenz, webbasiert, etc.)?
- Greift das Produkt aktiv in den Produktionsprozess ein, werden z.B. im Falle einer erkannten Anomalie Produktionsanlagen gestoppt?
 - o Inwiefern übernimmt Ihre Firma hierfür eine Garantie/Verantwortung oder wird die Entscheidung zum Produktionsstopp (und damit das Risiko) letztendlich dem Kunden überlassen?
- Können Daten aus dem System exportiert werden?
 - o Welche Schnittstellen werden für den Datenexport angeboten?
 - o In welchen Dateiformaten können die Daten exportiert werden?

Des Weiteren wurde den Herstellern im Anschreiben die Möglichkeit für die Durchführung einer einstündigen Telefonkonferenz eingeräumt.

Die folgende Tabelle fasst die Resonanz der Hersteller im Hinblick auf die Beantwortung des Fragebogens zusammen. Es wurde vermerkt, ob ein ausgefüllter Fragebogen vom jeweiligen Hersteller zurückgesendet wurde und ob eine Telefonkonferenz stattgefunden hat.

Hersteller	Fragebogen ausgefüllt	Telefonkonferenz stattgefunden
Checkpoint	Ja	Ja
Cisco	Nein	Nein
Gateprotect	Nein	Nein
Infoblox	Ja	Ja
Juniper	Nein	Nein
Lancope	Nein	Nein
Tofino Security	Nein	Nein

Die Auswertung der Herstellerrückmeldung erfolgt im nächsten Abschnitt.

1.8.2 Auswertung

Bei den durchgeführten Telefonkonferenzen war es möglich, über den Fragebogen hinausgehende Punkte mit den jeweiligen Firmenrepräsentanten zu diskutieren. Diskutiert wurden die 1200R Rugged Appliance der Firma Checkpoint und die Produktserie DNS Secure der Firma Infoblox (siehe Abschnitt 5.2) mit den jeweiligen Repräsentanten der beiden Firmen. Die weiterführenden Fragen befassten sich vor Allem mit der Möglichkeit zum Einsatz in einem Industrie 4.0-Paradigma.

Schwerpunktthema hierbei war der Einsatz der jeweiligen Lösungen in einer Fertigungslandschaft, in der Ende-zu-Ende-Verschlüsselung auf Shopfloorebene umgesetzt ist. Bei der 1200R Rugged Appliance kann dies durch das Einbringen zusätzlicher Zertifikate umgesetzt werden. Die Verschlüsselung würde dann an dieser Stelle aufgebrochen werden. DNS Secure zielt auf die Absicherung von DNS ab, Ende-zu-Ende-Verschlüsselung ist von daher kein relevantes Szenario.

2 Entwicklung und Integration von Methoden zur netzwerkseitigen Anomaliedetektion

Zur Erkennung von Anomalien werden im Rahmen des Projekts Anomaliedetektionswerkzeuge erzeugt. In diesem Abschnitt werden diese vorgestellt.

2.1 TUM Anomalieerkennungswerkzeug

Name des Tools:	TUM Anomalieerkennungswerkzeug
Beschreibung:	1- For preprocessing part, we get help from Wireshark and Tshark for capturing and analyzing the pcap files. First we have to capture the network traffic, saving the pcap files and then giving the pcap into our IPA to extract the intended features as an output. This output could be a json or csv file format. 2- For the learning part, we are now developing our Machine Learning framework covering with different range of ML algorithms with Python language. We take the output from IPA framework then feed into our ML models to classify/detect normal from malicious activity. This module classifies traces of the pcap files and can send the feedback to the security expert.
Benötigter Input:	Netzwerkverkehr
Generierter Output:	Anomalielevel der überwachten Verbindung
Benötigte Hard-, Software:	- Für den Prototypen: Raspberry Pis - Tshark - Python

Steuerungssysteme moderner Industrieanlagen (industrial control systems, ICS) stellen einen reibungslosen Ablauf der Fabrikautomatisierung und Prozesssteuerung sicher. Für die innere Sicherheit eines Landes ist es von wesentlicher Bedeutung, dass die vielfältigen Anomalien mitunter kritische Infrastrukturen (KRITIS) betreffen.

Die Komponenten solcher Kontrollsysteme haben sehr lange Lebenszyklen von etwa 20 Jahren oder mehr. Dementsprechend hoch sind die Anforderungen an die Kompatibilität zwischen vorhandenen und neuen Systemen. Vor diesem Hintergrund beruht eine Vielzahl

der verwendeten Protokolle, die zur Kommunikation zwischen Sensoren, Peripherie und Steuerungen verwendet werden, auf überholten Definitionen und Standards.

Darüber hinaus erhöht der stark ansteigende Grad der Vernetzung der Komponenten und intensiven Nutzung von Informationstechnologie das Risiko eines erfolgreichen Cyberangriffs mit erheblichen Folgen für das wirtschaftliche und soziale Wohlergehen der Bevölkerung.

Zum Schutz gegen ein erfolgreiches Eindringen in ICS entwickeln wir im Kontext unserer Forschung einen Rahmen zur **Erkennung von Anomalien im industriellen Netzverkehr** (network intrusion detection). Im Vordergrund steht die Realisierung der Module zur Analyse des allgemeinen Netzverkehrs und zur Inspektion einzelner Pakete.

Die Untersuchung großer Datenströme und eine präzise Klassifizierung von Bedrohungen werden dabei durch die Implementierung diverser Algorithmen des maschinellen Lernens gestützt. Entscheidend für eine exakte Analyse sind neben der Verfügbarkeit realistischer Datensätze die gut ausgearbeitete Parametrisierung der Algorithmen bezogen auf den jeweiligen Anwendungsfall. Zu diesem Zweck wird das System sowohl mittels bereitgestellter Echtzeit-Daten von den industriellen Kooperationspartnern als auch mittels eigens erzeugter Netzdaten trainiert.

Das Modul zur Echtzeit-Analyse des Netzverkehrs bedient sich den Funktionen von TShark, einer auf der Kommandozeile basierten Version von Wireshark. Die bisherige Implementierung unterstützt die Extraktion von Paketen und deren Felder basierend auf den Protokollen PROFINET und S7¹.

Bedrohungen werden in einem weiteren Modul durch die Erkennung von Anomalien festgestellt und klassifiziert. Das Ziel ist, neben der eigentlichen Erkennung eines Versuches des Eindringens, die präzise Bestimmung der Art des Angriffs. Hier geht man von einem gewissen Normwert aus, der sich aus dem Inhalt und der zeitlichen Abfolge jeweiliger Pakete im Netz ergibt. Falls das System eine statistisch signifikante Abweichung dieses Werts feststellt, liegt der Verdacht auf einen Angriff nah. Eine solche Meldung erfordert daraufhin weitere Analysen zur Feststellung, ob ein Angriff stattfindet.

Unsere Implementierung unterstützt die Verwendung sogenannter überwachter (*supervised*) und unüberwachter (*unsupervised*) Algorithmen. Die überwachten Verfahren müssen zunächst mittels fester Trainingsdaten initialisiert werden, um aussagekräftige Klassifizierungen vorzunehmen. Unüberwachte Methoden sind hingegen flexibler, als dass sie für gegebene Eingaben in der Lage sind, selbstständig ein Modell zu erstellen und anhand dieses Anomalien zu erkennen.

¹ Verwendet von Siemens SIMATIC-Komponenten.

Die Erzeugung von Netzverkehrsdaten ist im Allgemeinen sehr schwierig, und die von der Industrie gelieferten Daten halten sich in Grenzen. Aus diesem Grund verwenden wir sogenannte **Gauß-Prozesse** (GP). GP sind generische, überwachte Verfahren maschinellen Lernens, welche zum Lösen von Regression- und probabilistischen Klassifizierungs-Problemen entwickelt wurden. GP können insbesondere im Rahmen einer One-Class-Classification (OCC) genutzt werden, die eine einfache Klassifizierung in übliche Daten vs. Ausreißer vornimmt. Der Vorteil eines GP ist die Eigenschaft, Vorhersagen gut zu interpolieren und im Rahmen von zu großen Abweichungen davon seine weiteren Voraussagen anzupassen. Nachteilig ist lediglich die Abhängigkeit von allen Sample/Features sowie die geringe Effizienz bei hoher Anzahl der Features.

Die Implementierung von **One-Class Support Vector Machine** (OC-SVM) gleicht den obigen Nachteil wieder aus. Auch diese Methode verwendet stets eine Klassifizierung, unterscheidet jedoch mittels Vektoren ob Daten "böartig" (*in-class*) oder "nicht böartig" (*outlier*) sind.

OC-SVM sind fähig, selbst eine Grenze zu erzeugen anhand der entschieden wird, ob sich neue Daten noch innerhalb derer befinden oder nicht. Sehr variable Angriffe, deren Muster auch noch nicht bekannt sind, können mit der besten Leistung aller One-Class-Trainings-Algorithmen erkannt werden. Einzig die Empfindlichkeit gegenüber der Parameter- und Kernausswahl ist ein entscheidender Nachteil und führt gegebenenfalls zur Intransparenz von Ergebnissen.

Darüber hinaus verwenden wir den **LOF-Algorithmus**, der ganz ohne eine Kern-Funktion auskommt und schließlich unempfindlicher als OC-SVM ist. Jedem Datenpunkt wird ein sogenannter LOF-Wert zugeordnet, der sich aus der Dichte seiner Nachbarn zusammensetzt. Kommen in einem Datenset viele ähnliche oder identische Punkte vor, so ist deren Dichte höher als die eines abweichenden Punktes. Dieser wird folglich als Anomalie erkannt.

Durch seinen lokal-basierten Ansatz werden mittels LOF Ausreißer erkannt, die von anderen Algorithmen als normal eingestuft werden. Dies geschieht auf Kosten der Rechenintensität und der Unfähigkeit, Aussagen über einen einzelnen Datenpunkt im Verhältnis zum gesamten Datenset treffen zu können.

2.2 Datenextraktion und Parsing (Fraunhofer SIT)

Name des Tools:	P4 - www.p4.org mit SIT-Erweiterungen
Beschreibung:	P4 ist eine Parser- und Paketverarbeiterbeschreibungssprache, aus der man bspw. ein Switch oder Router kompilieren kann. Für die Anomalieerkennung nutzen wir den Teil der Sprache der Parser erzeugt und generieren damit Rohdatenoutput. Für die TCP-basierten Protokolle wurde ein zusätzlicher Stream-Reassembler implementiert, der die TCP-Segmente reassembliert und diese dann wiederum in einen P4-generierten Parser schickt. Zusätzlich arbeiten wir gerade an einer Analysesprache, die aus den Parser-Daten dann Zustände ableiten soll. Zwischen diese Zuständen lassen sich gültige Übergänge definieren, die den Normalbetrieb modellieren. Abweichungen stellen dann zu detektierende Anomalien dar.
Benötigter Input:	- Pakete (via PCAP oder direkt von der Netzwerkkarte) - P4-Parser-Beschreibung der eingesetzten Protokolle, bspw: <pre>header_type ethernet_t { fields { dstAddr : 48; srcAddr : 48; etherType : 16; } }</pre> - optional Zustandbeschreibungen (deviceId = 1) and (data = 1) => LichtschrankeA
Generierter Output:	- Header->Feld->Wert Zuordnungen, in XML bspw. (stark verkürzt!): <pre><packet number="441"> <ethernet> <dstAddr type="hex">90A2DA10D8B7</dstAddr> <srcAddr type="hex">B827EB712379</srcAddr> <etherType type="hex">0800</etherType> </ethernet> </packet></pre> - Zustände: LichtschrankeA = 0 - Zustandsübergänge: LichtschrankeA löst aus => MotorB springt an - Anomalien: MotorB springt an, obwohl LichtschrankeA nicht ausgelöst hat
Benötigte Hard-, Software:	- Irgendwas mit Ethernet auf dem die P4 Software-Implementierung läuft - bspw. Standard-PC mit aktuellem Ubuntu

Ziel der hier beschriebenen Methode ist das Extrahieren der Befehlsdaten einer Steuerung an die mit ihr verbundenen Komponenten aus Bussystemen, die auf Ethernet-Technologie aufbauen.

Dazu werden zwei Ansätze vorgestellt, die Datenpakete aus Ethernet-Verbindungen extrahieren können: Die klassische physikalische Extraktion mit Splittern (Taps) und eine auf Software Defined Networking (SDN) basierende. Die so gewonnen Datenpakete sollen aufbereitet werden durch Hinzufügen von Metadaten (beispielsweise Zeitstempel, Bezeichnung der Empfangsquelle wie Switch- und Portnummer, Senderichtung). Diese angereicherten Pakete können anschließend verschiedenen Analyswerkzeugen zu Verfügung gestellt werden, etwa über das PCAP-Format [15].

Eines dieser Werkzeuge wird ebenfalls vorgestellt. Es ist eine Sammlung an Parser-Beschreibungen für verschiedene Protokolle, die die extrahierten Pakete in ihre Datenfelder aufteilen kann. Diese Sammlung ist in P4 [16], einer domänenspezifischen Programmiersprache (Domain specific language, DSL) implementiert. Einige benötigte Erweiterungen wurden in die Sprache integriert, diese werden ebenfalls kurz vorgestellt.

2.2.1 Untersuchte Plattformen

Um zu gewährleisten, dass die entwickelten Extraktions- und Verarbeitungsmethoden universell einsetzbar sind, wurden sie an zwei Plattformen getestet:

2.2.1.1 Fraunhofer SIT Demonstrator

Der Fraunhofer SIT Demonstrator besteht aus einem einfachen Pumpen-Regelkreis. Ziel der darin integrierten Industrieanlage ist es, den Wasserstand in einem Zylinder zwischen zwei konfigurierbaren Maximal- und Minimalwerten zu halten.

Dazu wird die Software-Steuerung OpenPLC [17] eingesetzt. Diese kommuniziert über Modbus [18] mit einer Pumpe sowie einem Ultraschallsensor für die Wasserstandserfassung. Die Kommunikation findet dabei über einen SDN-Switch statt, der über einen Pox-Controller [19] gesteuert wird.

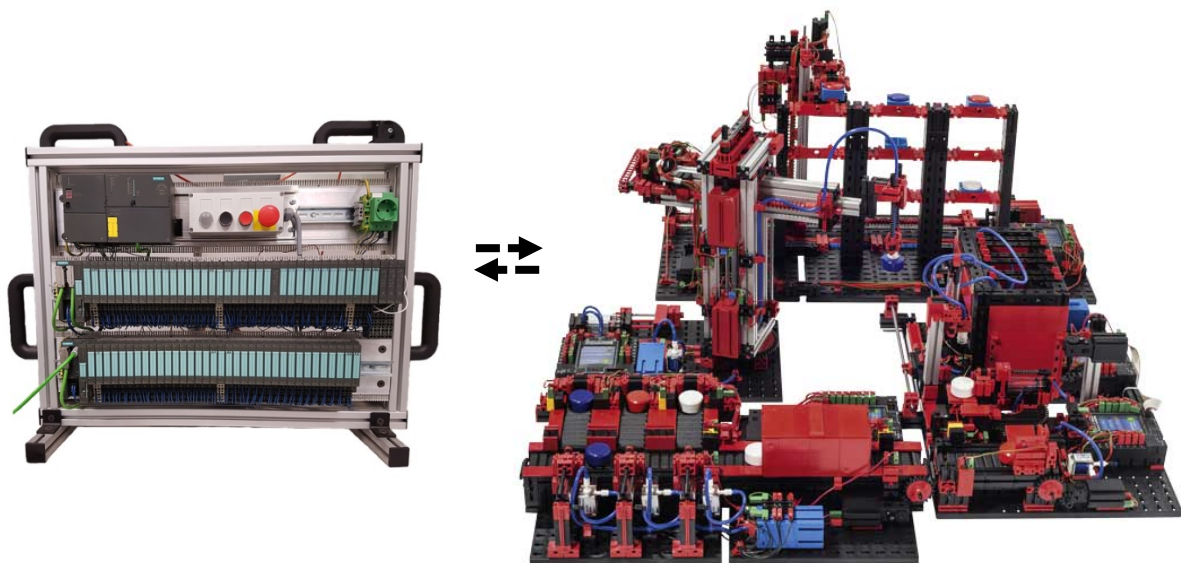


Dieser Demonstrator wurde inklusive den hier im Weiteren beschriebenen Extraktions- und Erkennungsmethoden von der Fraunhofer Gesellschaft auf der Hannover Messe 2017 ausgestellt.

2.2.1.2 AP4 Demonstrator

Beim zweiten Demonstrator handelt es sich um den in Arbeitspaket 4 zu erarbeitenden Demonstrator, an dem abschließend die entwickelten Methoden gezeigt werden sollen. Der Demonstrator zeigt eine Modellfabrik, die verschiedene Bearbeitungsschritte abarbeiten kann (Entnahme von Türrohlingen aus dem Hochregallager über ein Regalbediengerät, interner Transport mittels Transfersystem und Förderband, Erzeugung von Türausschnitten in der Laserschneidanlage, Entgraten, Vermessen, Sortieren und Wiedereinlagern von fertigen Autotüren). Die hohe Komplexität dieses Demonstrators bildet die Grundlage zur Evaluierung der entwickelten Methoden. So kann gezeigt werden, dass die entwickelten Methoden auch in der zukünftigen Produktion einsatzfähig sind.

Die Steuerung der Modellfabrik übernimmt eine Siemens Simatic S7 300. An ihr sind Eingabe- und Ausgabemodule über einen PROFINET-Feldbus [20] angebunden. Zur Extraktion der Feldbuskommunikation werden aktive Taps verwendet.



Dieser Demonstrator wurde ebenfalls auf der Hannover Messe 2017 gezeigt, dort jedoch noch ohne die Integration der entwickelten Methoden.

2.2.2 Datenextraktion durch Taps

Taps sind Geräte, die sich in bestehende Kommunikationsstrecken einbinden lassen, und Kopien der ausgetauschten Dateien ausleiten können. Für die in den Demonstratoren eingesetzten Ethernet-Verbindungen existieren zwei Arten von Taps:

- **Passiv:** Die elektrischen Signale werden direkt ohne Verarbeitung an einem weiteren Anschluss zu Verfügung gestellt. Gängige Ethernet-Hardware ist problemlos in der Lage, genügend Sendeleistung für zwei Empfänger zu Verfügung zu stellen wenn in Bussystemen übliche Kabellängen verwendet werden.

Vorteile des passiven Taps ist, dass keinerlei Einfluss auf die Signale genommen wird (abgesehen vom erhöhten Energieverbrauch). Dadurch findet keine Signalverzögerung statt, was in Echtzeitsystemen vorteilhaft ist.

- **Aktiv:** Der aktive Tap terminiert die Ethernet-Verbindung beidseitig, puffert die empfangenen Pakete und sendet sie jeweils einmal am regulären Ausgang als auch am Monitorausgang ab. Der Tap benötigt dafür eine eigene Stromversorgung und entlastet so die Sender, da diese nicht die zusätzliche Leistung für den Empfang am Monitorport aufbringen müssen.

Vorteile des aktiven Taps sind die Fähigkeit, Gigabit-Verbindungen [21] unterstützen zu können. Diese sind mit passiven Taps nicht beobachtbar, da bei diesen die Datenleitungen gleichzeitig zum Senden und Empfangen genutzt werden.

Da für viele Anwendungen eine Fast-Ethernet-Verbindung ausreicht, und manche Bussysteme nicht für Gigabit-Ethernet spezifiziert sind, wie PROFINET, SERCOS III [22] und EtherCAT [23] ist der passive Tap insbesondere für zeitkritische Anwendung erste Wahl.

2.2.3 Datenextraktion mit Software Defined Networking (SDN)

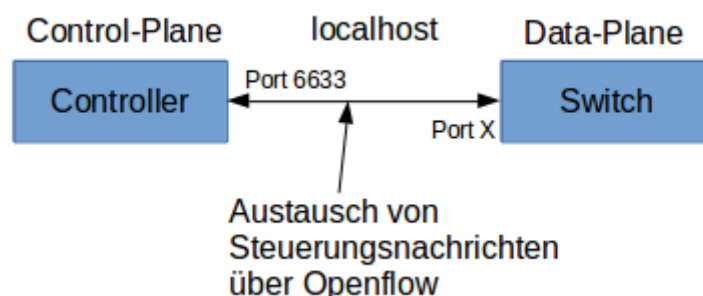
Moderne Industriesysteme werden in kommenden Industrie 4.0 Szenarien deutlich stärker vernetzt sein als heute. Eine Schlüsseltechnologie für die flexible Vernetzung komplexer Strukturen ist dabei Software Defined Networking. Dieser Abschnitt beschreibt die Datenextraktion aus SDN-fähigen Industrienetzen.

Vorteile gegenüber des Einsatzes von Taps oder anderen Monitoring-Lösungen ist hier, dass bereits bestehende Switch-Infrastruktur genutzt werden kann. Es ist also kein Hardware-Einbau nötig, und eine Sicherheitslösung ließe sich bedarfsorientiert und zeitnah einsetzen. SDN-basierte Datenextraktion ist unabhängig vom eingesetzten Übertragungsprotokoll der physikalischen Schicht und damit tauglich für heutige und künftige Gigabit-Netze. Weiterhin erlaubt SDN eine Vorfilterung beziehungsweise Auswahl der zu beobachtenden Datenpakete und kann so die Monitoring-Lösung entlasten, da dort nur relevante Daten anfallen.

Die Nachteile gegenüber Taps sind der zusätzliche Aufwand beim Betrieb und der Sicherung der SDN-Infrastruktur, sowie die Verzögerungen beim Switching, die bei zeitkritischen Anwendungen relevant sein könnten.

2.2.3.1 Kurzbeschreibung Software Defined Networking

Software Defined Networking ermöglicht es dem Nutzer, seine Netzwerkkomponenten mittels Software zu steuern und zu konfigurieren. Typisch für SDN ist die Trennung in Control-Plane und Data-Plane:



Die Control-Plane, die die Behandlung eingehender Pakete bestimmt, befindet sich im Controller des Netzwerkes. Hier werden die Netzwerkdaten mit Hilfe von Software geparkt und analysiert. Nach diesem Schritt entscheidet die Software des Controllers, welche Steuerungsregeln für die Netzwerkgeräte erzeugt werden.

Empfänger dieser Steuerungsregeln, sind Hubs, Switches oder Router, welche nur noch aus einer Data-Plane bestehen, die für die Weiterleitung der Pakete anhand dieser Regeln verantwortlich ist. Empfängt ein solches Gerät ein Datenpaket aus dem Netzwerk, so durchsucht es seine interne Forwarding-Tabelle nach einer geeigneten Regel. Ist keine Regel vorhanden, so wird das Datenpaket an den Controller geschickt. Der Controller teilt dem Gerät in einer Nachricht mit, welche Steuerungsregel für das eingetroffene Paket genutzt werden soll. Je nach empfangener Steuerungsregel ist diese einmalig anzuwenden oder aber dauerhaft für den Pakettyp und damit in der Forwarding-Tabelle abzulegen.

2.2.3.2 OpenFlow 1.0

Mittel des OpenFlow Protokolls wird die Kommunikation zwischen Controller und Netzwerkgerät realisiert. Nachfolgend werden die relevanten OpenFlow Nachrichten behandelt.

PacketIn:

Name	Bits	Byte Ordering	Constraints
buffer_id	32	MSBF	none
total_len	16	MSBF	none
in_port	16	MSBF	none
reason	8	-	See Table
pad	8	-	none
data	48	MSBF	none

Bild: PacketIn Quelle: flowgrammable.org

Eine PacketIn Nachricht wird von den Netzwerkgeräten erzeugt, wenn sie keinen geeigneten ‚Match‘ in der Forwarding-Tabelle finden. Jetzt wird die empfangene Nachricht in einer PacketIn Nachricht verpackt und an den Controller zur Analyse geschickt. Dieser extrahiert die Nachricht aus dem ‚Data‘ Bereich der PacketIn Nachricht.

Zusätzlich wird in der PacketIn Nachricht über ‚Reason‘ mitgeteilt, ob die Nachricht aufgrund eines nicht vorhandenen ‚Match‘ versendet wurde oder ob die Nachricht, aufgrund einer Extraktionsregel an den Controller geschickt wurde.

FlowMod:

Name	Bits	Byte Ordering	Constraints
match	32	MSBF	match restrictions
cookie	64	MSBF	!= 0xFFFFFFFFFFFFFFFF
command	16	MSBF	See below
idle_timeout	8	-	none
hard_timeout	8	-	none
priority	8	-	none
buffer_id	32	MSBF	none
out_port	16	MSBF	none
flags	16	MSBF	See below
action[]	32	MSBF	action restrictions

Bild: Flowmod Quelle: flowgrammable.org

FlowMod ist die vom Controller erzeugte Nachricht mit der Steuerungsregel. Wurde ein Netzwerkpaket analysiert, wird je nach Ergebnis eine geeignete FlowMod Nachricht erzeugt. Diese sendet der Controller anschließend an das Netzgerät. Das Herzstück dieser Nachricht, ist der ‚Match‘ und ‚Action‘ Eintrag, die die Regeln zur Weiterleitung enthalten.

Action/Match:

Name	Bits	Byte Ordering	Constraints
wildcards	32	MSBF	See below
in_port	16	MSBF	0x0000...0xfeff
dl_src	48	MSBF	802.3 MAC
dl_dst	48	MSBF	802.3 MAC
nw_src	32	MSBF	IPv4 Source
nw_dst	32	MSBF	IPv4 Destination
tp_src	16	MSBF	TCP/UDP Source Port
tp_dst	16	MSBF	TCP/UDP Destination Port

Bild: Match Quelle: flowgrammable.org

Erhält das Netzwerkgerät eine FlowMod Nachricht, so wird der ‚Match‘ und ‚Action‘ Eintrag des Paketes zusammen in der Forwarding-Tabelle des Gerätes gespeichert.

Jedes empfangene Netzwerkpaket wird mit den vorhandenen ‚Match‘ Einträgen verglichen. Als Kriterien kann ein ‚Match‘ z.B. Quell- oder Ziel-IP-Adresse, Protokolltyp oder andere Einträge enthalten (siehe Bild). Findet das Gerät ein Netzwerkpaket, das alle Kriterien eines Matches erfüllt, so wird die dazugehörige Action ausgeführt.

Type Name	Type Value	Field Name	Bits	Byte Ordering	Constraints
Output	0x0000	type	16	MSBF	= 0x0000
		length	16	MSBF	= 0x0008
		port	16	MSBF	in 0x0000..0xffef
		max_len	16	MSBF	none

Bild: Output

Quelle: flowgrammable.org

In einer ‚Action‘ kann z.B. angegeben sein, dass ein ‚Output‘, also ein Weiterleiten, über einen ‚Port‘ stattfinden soll.

2.2.3.3 Genutzte Hardware und Software

In diesem Teil werden die verwendeten Hard/Software Komponenten vorgestellt, welche benutzt werden, um die SDN-basierte Extraktion zu realisieren.

Switch

Es wird ein ‚Zodiac FX‘ von Northbound genutzt. Dieser Switch besitzt 4 Fast Ethernet Ports und wird über eine USB-Verbindung konfiguriert. Openflow 1.0, 1.3 und 1.4 werden unterstützt.

Controller

Als Controller wurde POX gewählt, welcher in Python implementiert ist und vorgefertigte Software bietet, die ein schnelles Aufsetzen eines einfachen Switches oder Hubs ermöglicht.

2.2.3.4 Beschreibung der Implementierung

Die folgende Netzwerktopologie dient zur Veranschaulichung der SDN-Architektur und zur Erklärung seiner Arbeitsweise.

In diesem Netzwerk, sind die Modbus-Geräte an den Zodiac-Switch angeschlossen. Über Switchport 1 bis 3 können die angeschlossene Geräte ihre Nachrichten versenden. Der Controller ist mit dem über Switchport 4 verbunden, über diesen Port erhält auch der Parser der später beschriebenen Analysesoftware seine Daten.

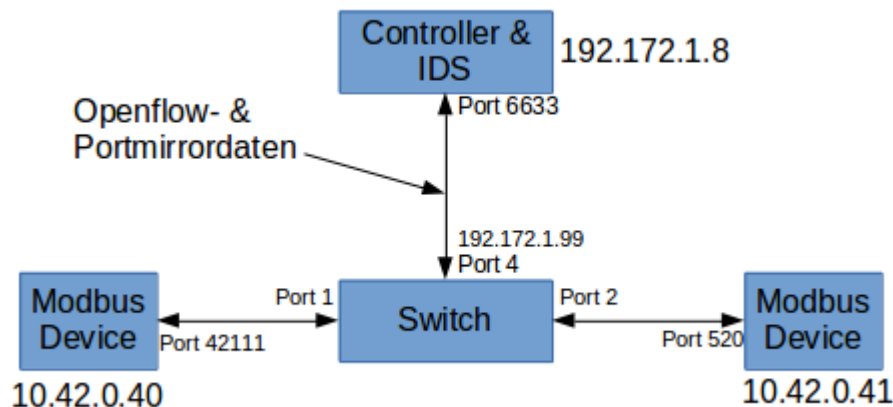


Bild: Netzwerktopologie

Um die benötigten Daten zu extrahieren wird ein Portmirror mittels SDN erstellt. Die vordefinierte Software des Controllers erzeugt nur ein einfaches Layer 2 Switch. Dazu muss das Controller-Programm erweitert werden.

Der Code des Controllers wird so erweitert, dass seine erzeugten FlowMod Nachrichten eine zusätzliche ‚Action‘ enthalten. In der ursprünglichen Version ist nur eine ‚Action‘ enthalten, welche dem Switch mitteilt, über welchen Port weitergeleitet werden soll. Die zusätzliche ‚Action‘ teilt dem Switch mit, dass jede Nachricht, die weitergeleitet wird, auch auf Port 4 gespiegelt werden soll.

Beispiel in Pseudocode:

```

FlowMod.match = of.create_match_from_input_packet(inputPacket)
FlowMod.actions.add( output(port = inputSwitchPort))
FlowMod.actions.add( output(port = 4))
send(FlowMod)

```

2.2.3.5 Ergebnis

Abschließend wurden Switch und Controller in den Fraunhofer SIT Demonstrator eingebaut. Die implementierte Software erzeugt wie gewünscht einen Mirrorport für den gesamten Datenverkehr. Das nachgelagerte Analysesystem, das später vorgestellt wird, kann die implementierte Anlage zuverlässig überwachen.

2.2.4 Parsing durch Paketbeschreibungssprachen

Die in Arbeitspaket 4.1 durchgeführte Marktanalyse zeigt eine große Menge an im Einsatz befindlicher Protokolle bei einem generellen Mangel an Parsern. Fast alle kommerziellen oder wissenschaftlichen Ansätze versuchen hier maschinelles Lernen einzusetzen, um

diesen Mangel zu kaschieren, in der Hoffnung, dass sich die Systeme mithilfe von Lernalgorithmen selbst programmieren.

Auffällig ist das Fehlen von stichhaltigen Nachweisen, die den erfolgreichen Einsatz in realen Systemen dokumentieren.

Daher fokussieren sich die hier vorgestellten Arbeiten auf die Entwicklung von Parsern und zeigen eine Möglichkeit auf, wie sich mit Hilfe von domänenspezifischen Sprachen (DSL) die Entwicklung von Parsern deutlich vereinfachen und beschleunigen lässt. Nicht zuletzt ist eines der Vorteile beim Einsatz von DSL, dass die Sprachen übliche Sicherheitslücken (wie etwa Buffer Overflows) und Designfehler verhindern können.

Die folgende Übersicht zeigt die im Rahmen der Arbeiten analysierten Sprachen und deren Einsatzfähigkeit im Industriekontext. Anschließend folgt ein Überblick über die für die Implementierung genutzte Sprache P4.

2.2.4.1 Übersicht über existierende Forschungsprojekte

Die folgende Auflistung stammt aus der im Rahmen der IUNO-Projektes abgeschlossenen Masterarbeit „Parsing and Analysis of Industrial Protocols to Build Real-Time Anomaly Detection Systems“ (**WORKING TITLE**) von Shubham Bhardwaj [24].

BinPAC (2006): Ein Werkzeug aus dem Intrusion Detection System (IDS) Bro [25]. Es ist fokussiert auf Protokolle in der Applikationsschicht und effiziente Arbeitsweise. Die Spezifizierung neuer Protokolle ist aufwändig, und es ist stark an das dazugehörige IDS gebunden. [BinPAC]

Spicy (2014): Nachfolger von BinPAC mit Fokus auf Deep Packet Inspection und Datei-Analyse. Im Gegensatz zu BinPAC lässt es sich ohne Bro nutzen. Es befindet sich jedoch noch in Entwicklung. [26]

NAIL (2015): Definiert eine Grammatik, mit der eingelesene Daten in ein Objektmodell transferiert werden. Dieses Modell ist in der Lage komplexe Formate wie Prüfsummen und Kompression abzubilden. Es ist reversibel, kann also auch zum Erzeugen von Paketen genutzt werden. [27]

CLICK (2000): Bietet eine modulare Architektur um Paketprozessoren zu erzeugen. Beschränkt sich auf Software-Implementierungen. Wird nicht mehr weiterentwickelt. [28]

Frenetic (2010): Eine Hochsprache für das Design von Switches. Motivation für das Projekt waren Limitierungen in OpenFlow und den dazugehörigen Controllern. Daher ist die Flexibilität gering im Vergleich zu anderen Lösungsansätzen. [29]

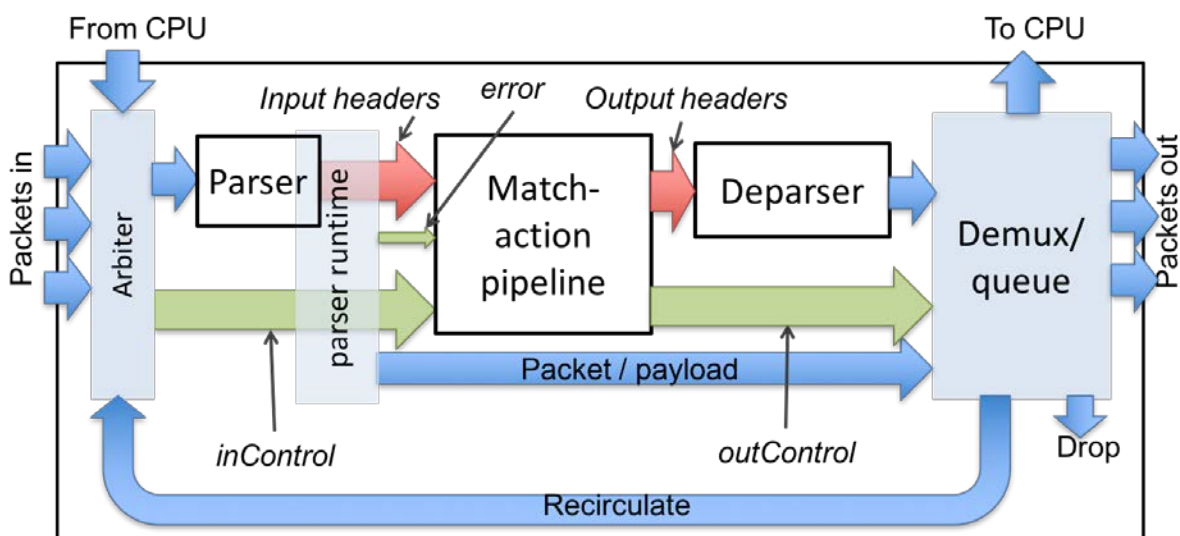
GAPAL (2007): Bietet generische Analyse von Protokollen auf Applikationsschicht. Das Parsing ist stark gekoppelt an die Analyse. Anweisungen werden interpretiert, die Ausführungsgeschwindigkeit ist vergleichsweise niedrig. [30]

P4 (2013): Protokollunabhängige Paketbehandlungssprache mit Fokus auf effizienter Verarbeitung großer Datenmengen. Alleinstellungsmerkmal ist die Fähigkeit auch für Hardware-Plattformen wie FPGAs und ASICs kompilieren zu können. [16]

2.2.4.2 Funktionsweise von P4

Die Hauptargumente zur Nutzung von P4 sind die sehr aktive Entwicklergemeinschaft sowie die universelle Einsatzfähigkeit auf verschiedenen Plattformen.

P4 entstand als eine Antwort auf die immer komplexer werdenden Versionen der OpenFlow Spezifikationen, die immer wieder neue Protokolle hinzufügten. Aus diesem Bedürfnis nach protokollunabhängigem Paketverarbeiten entstand die Sprache, die in der Lage sein sollte, zukünftige Netzwerk-Hardware programmierbar zu machen, und so Software Defined Networking in Richtung „Software Defined Switch“ zu erweitern.



Die Paketverarbeitung wird in P4 dabei in mehrere Schritte unterteilt:

Parse: Anhand der Header-Spezifikation wird das Paket in seine Felder zerlegt, so dass diese anschließend ausgewertet werden können.

Match: Die Felder werden verglichen mit Referenzwerten aus den Match+Action Tabellen. Diese Tabellen enthalten beispielsweise Zuordnungen wie MAC-Adresse an Port.

Action: Die in den Match+Action Tabellen gelisteten Aktionen werden ausgeführt, beispielsweise das Weiterleiten an das Zielgerät, Umschreiben von Adressen (NAT), oder Verwerfen (Firewall)

Deparse: Die bearbeiteten Pakete werden aus den Feldern wieder zusammengesetzt, falls die vorhergehenden Actions eine Modifikation durchgeführt haben.

Queue: Die ausgehenden Pakete werden an ihr in der Action-Phase bestimmtes Ziel gesendet, oder verworfen, falls keines bestimmt wurde.

Für Arbeitspaket 4 ist hier nur der erste Schritt relevant, da die in P4 spezifizierten Aktionen hauptsächlich für das Weiterleiten von Daten konzipiert sind. Die folgenden Abschnitte zeigen, wie sich P4 mit wenigen Änderungen nutzen lässt um damit auch komplexere Protokolle auf höheren OSI-Schichten zu behandeln.

2.2.5 Einsatz von P4 zum Parsing von Feldbusprotokollen

Bei der Implementierung der in den Demonstratoren genutzten Protokollen wurden zusätzliche Änderungen an der P4 Software Sammlung vorgenommen, um die speziellen Anforderungen zu erfüllen. Diese werden in den folgenden Abschnitten behandelt

2.2.5.1 Unterstützung für Schleifen

Protokolle wie Profinet und LLDP nutzen Type-Length-Value (TLV) Listen, um mehrere Informationselemente in einem Paket zu versenden. Dazu müssen die TLV-Felder nacheinander abgearbeitet werden. P4 bietet dazu Array-Konstrukte an, um mehrere Felder gleichen Typs zu verarbeiten.

Ein Problem bei der Verarbeitung mehrerer Felder ist jedoch, dass diese nicht in Schleifen durchlaufen werden können, da der P4-Compiler nicht in der Lage ist, einen Deparser für einen Protokollbaum zu erstellen der potenziell nicht terminiert.

Das Problem ist den Entwicklern bereits bekannt und wird dort bearbeitet, so dass zukünftige Versionen von P4 damit umgehen können. In der Zwischenzeit behilft sich die jetzige Lösung mit dem Deaktivieren des Deparsers. Dies stellt momentan keine Einschränkung dar, da die Routinen zur Paketausgabe nicht benötigt werden.

2.2.5.2 Referenzen auf vorherige Header und Metadaten

Beim Einlesen von Profinet-Datenelementen fällt auf, dass die transportierten Nutzdaten kein Längenfeld aufweisen. Die Länge der Daten ist implizit berechenbar aus der Paketlänge. Da P4 kein Feldkonstrukt aufweist, welches alle Bytes bis zum Paketende extrahieren kann,

benötigt man zur Feldlängenberechnung die Gesamtzahl der Bytes in einem Paket. So lässt sich der Extraktor für Felder mit variabler Länge nutzen.

Die aktuelle Sprachdefinition erlaubt aber nicht, dass ein Header bei der Längenberechnung auf Felder in anderen Headern zugreifen kann. Diese Funktionalität wurde ergänzt, so dass bei der Berechnung der Nutzdatenlänge auf den Metadaten-Header zugegriffen werden kann, der die Paketlänge enthält.

2.2.5.3 TCP-Unterstützung

P4 ist ausgerichtet auf die Verarbeitung der unteren OSI-Schichten. Daher werden keine Inhalte von TCP-Verbindungen behandelt, sondern lediglich deren Header-Daten. Um TCP-basierte Protokolle wie Modbus zu unterstützen, wurde daher ein TCP-Reassembler implementiert, der die TCP-Pakete wieder zu Datenströmen zusammensetzt. Die zusammengesetzten Datenströme werden dann in die einzelnen Nachrichten unterteilt und in eine zweite P4-Instanz geleitet, die das innere Protokoll (etwa Modbus) behandelt.

2.2.5.4 Zeitstempel

Da im industriellen Kontext Prozesse stark zeitabhängig sind, werden allen eingehenden Paketen direkt nach dem Empfangen zusätzlich Zeitstempel hinzugefügt. Diese Zeitstempel befinden sich beispielsweise auch in PCAP-Aufzeichnungen.

Diese Funktionalität macht die Paketverarbeitung unabhängig von der Uhr des Computers und erlaubt die Auswertung zeitkritischer Prozesse aus Aufzeichnungen.

2.2.6 Beispielausgaben

Beispiel 1 - Ethernet:

Eingabedatei in P4

```
header ethernet_t ethernet;
header_type ethernet_t {
    fields {
        dstAddr : 48;
        srcAddr : 48;
        etherType : 16;
    }
}

parser start {
    return parse_ethernet;
}

parser parse_ethernet {
    extract(ethernet);
    return ingress;
}
```

Ausgabe als XML-Struktur eines Pakets:

```
<packet number="49">
  <standard_metadata>
    <ingress_port type="hex">0000</ingress_port>
    <timestamp type="hex">00054F3E45079492</timestamp>
    <packet_length type="hex">000003E9</packet_length>
    <parsed_bytes type="hex">000003E9</parsed_bytes>
  </standard_metadata>
  <ethernet>
    <dstAddr type="hex">000E8CACFCB4</dstAddr>
    <srcAddr type="hex">000E8CABC4DA</srcAddr>
    <etherType type="hex">0800</etherType>
  </ethernet>
</packet>
```

Beispiel 2 - Felder mit variabler Länge und Strings:

Ausschnitt aus Eingabedatei in P4:

```
header_type lldp_tlv_chassis_id_t {
  fields {
    tlv_type : 7;
    tlv_length : 9;
    subtype : 8;
    chassis_id : *;
  }
  length : tlv_length + 2;
  max_length : 1500;
}
```

Ausgabe (Ausschnitt):

```
<lldp_tlv_chassis_idnumber="0">
  <tlv_type type="hex">01</tlv_type>
  <tlv_length type="hex">0007</tlv_length>
  <subtype type="hex">07</subtype>
  <chassis_id type="string">pnohen</chassis_id>
</lldp_tlv_chassis_id>
```

Beispiel 3 - TCP Nachrichten und Modbus:

Ausschnitt aus Eingabedatei in P4:

```
header_type modbus_t {
  fields {
    trans_id : 16;
    proto_id : 16;
    msg_length : 16;
    unit_id : 8;
```



```

    }
}

header_type modbus_readdi_response_t {
    fields {
        function_code : 8;
        byte_count : 8;
        data : *;
    }
    length : byte_count + 2;
    max_length : 1500;
}

```

Ausgabe (Ausschnitt):

```

<tcp_message number="37">
  <meta_header>
    <client_ip type="hex">0A2A0028</client_ip>
    <server_ip type="hex">0A2A0029</server_ip>
    <client_port type="hex">A59C</client_port>
    <server_port type="hex">01F6</server_port>
    <from_server type="hex">01</from_server>
    <stray_session type="hex">00</stray_session>
    <reserved type="hex">00</reserved>
    <reference type="hex">0000000000000000A6</reference>
    <offset type="hex">00000000</offset>
  </meta_header>
  <modbus>
    <trans_id type="hex">0009</trans_id>
    <proto_id type="hex">0000</proto_id>
    <msg_length type="hex">0004</msg_length>
    <unit_id type="hex">01</unit_id>
  </modbus>
  <modbus_readdi_response>
    <function_code type="hex">02</function_code>
    <byte_count type="hex">01</byte_count>
    <data type="hex">01</data>
  </modbus_readdi_response>
</tcp_message>

```

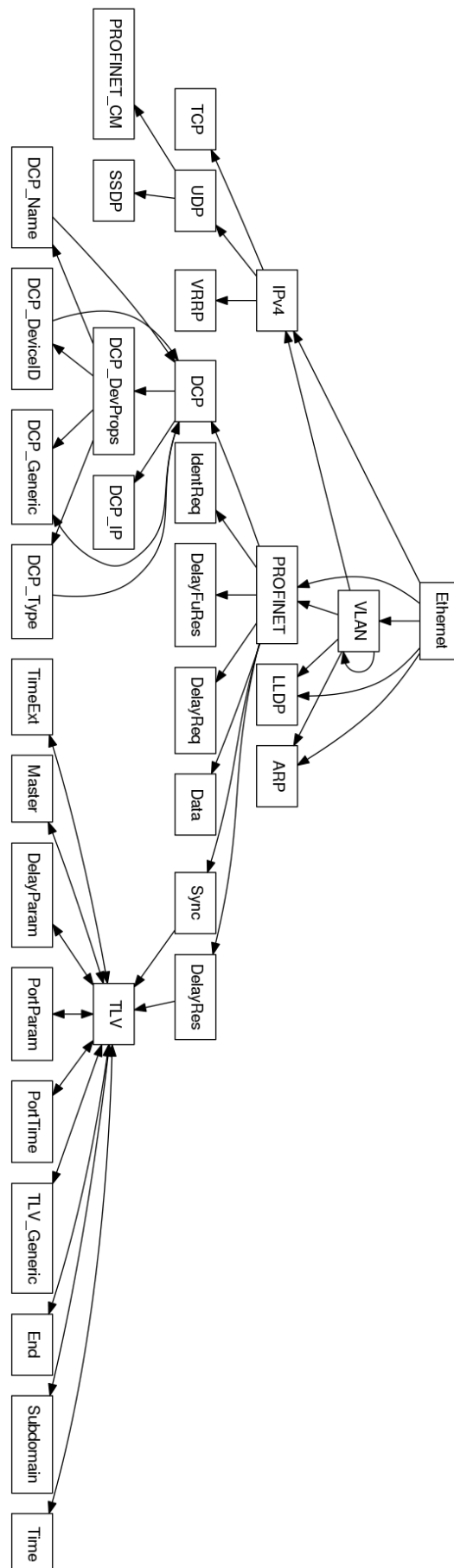
2.2.6.1 Verfügbare Implementierungen

Für folgende Protokolle wurde ein vollständiger oder teilweiser P4-basierter Parser implementiert:

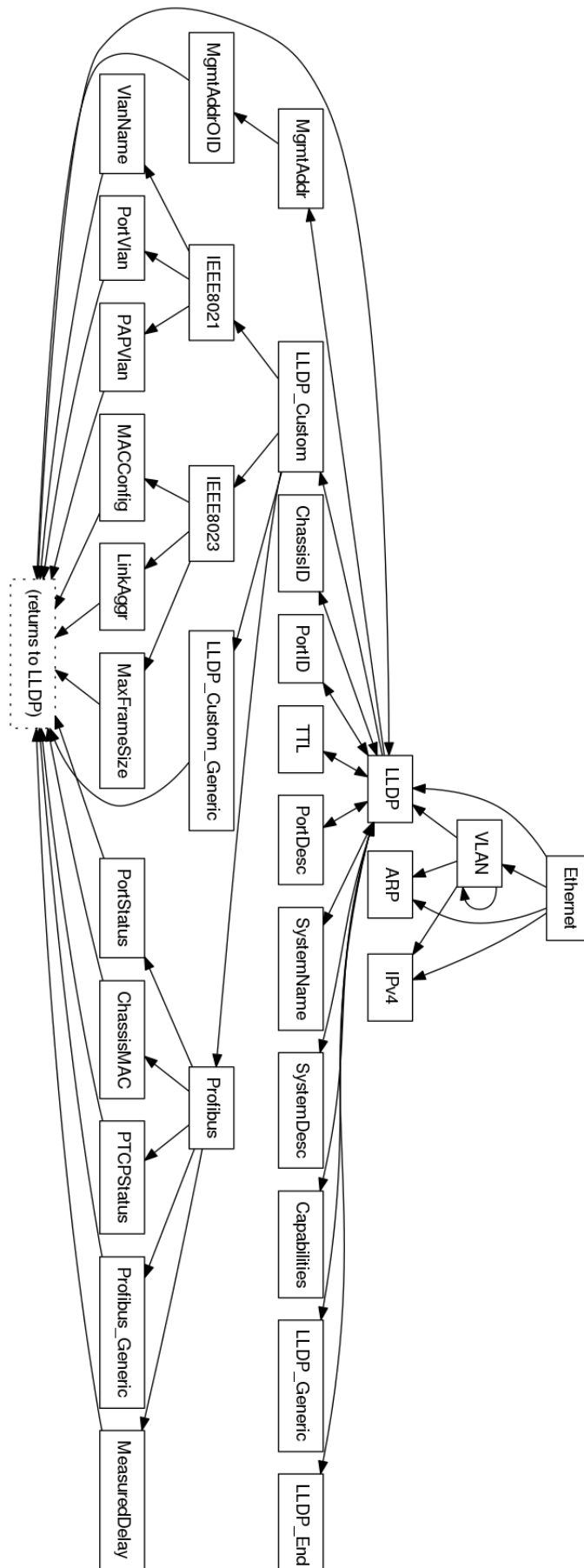
OSI-Layer	Protokoll	Umfang der Implementierung
Data Link (2)	Ethernet	vollständig
	ARP	teilweise
	VLAN	vollständig
	PROFINET	teilweise, einsatzfähig
	LLDP	teilweise, einsatzfähig
Network (3)	IPv4	vollständig
	VRRP	rudimentär
Transport (4)	TCP	teilweise, einsatzfähig
	UDP	vollständig
Application (7)	PROFINET-CM	rudimentär
	Modbus	teilweise, einsatzfähig
	OPCUA	rudimentär

Die folgenden Grafiken zeigen den Aufbau der Parserbäume und die unterstützten Feldtypen.

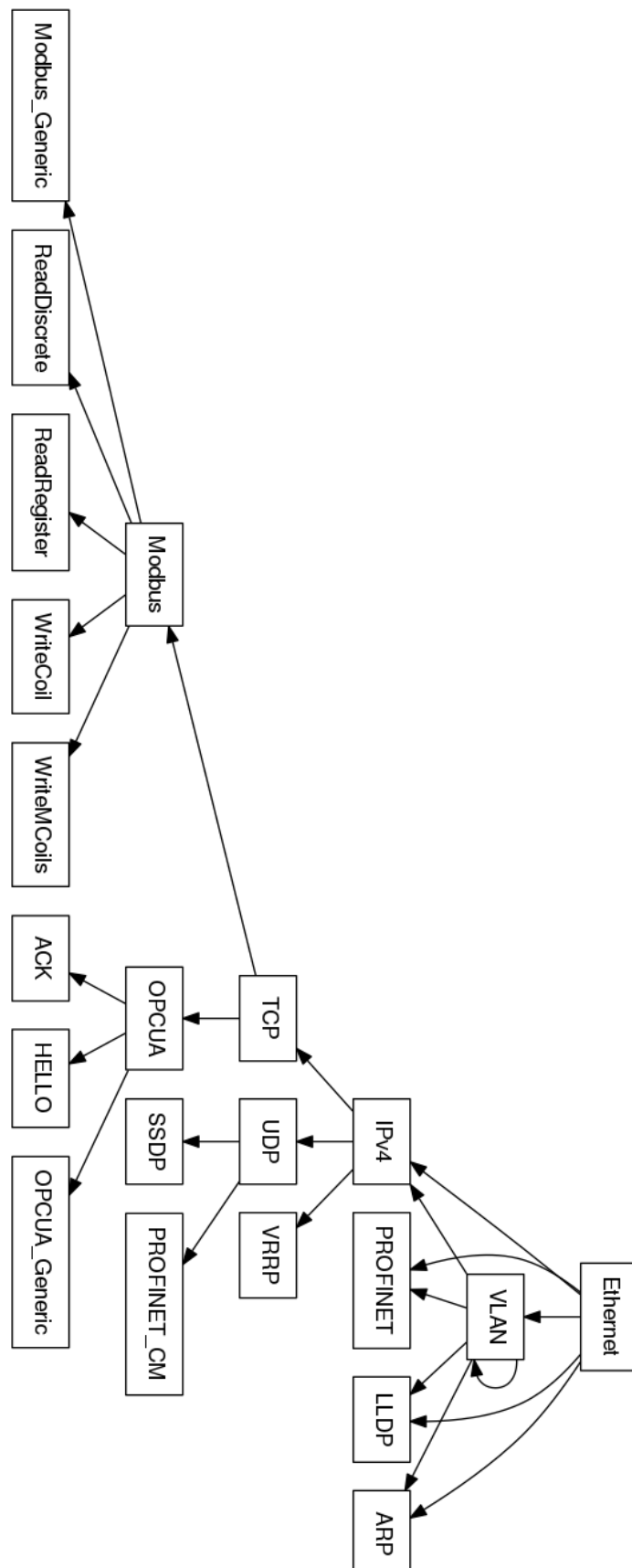
Parsergraph PROFINET



Parsergraph LLDP



Parsergraph Modbus, OPCUA



2.3 Interpretation der extrahierten Daten und Fehlererkennung durch Simulation (Fraunhofer SIT)

Das Ziel der hier beschriebenen Methode ist die Erkennung von Fehlverhalten anhand der im vorherigen Schritt gewonnenen Daten. Diese Daten enthalten vereinfacht gesehen Tupel der Form Adresse → Wert. Die Adresse ist dabei ein komplexer Typ, der aus mehreren Spezifikatoren besteht. Beispiele hierfür sind:

Modbus: IP-Adresse, Port, Slave ID, Nummer des gelesenen/geschriebenen Registers

Profinet: MAC-Adresse, Frame ID, Slot und Subslot

Die Adresse lässt sich eindeutig einem Sensor oder Aktor zuordnen, und die dort geschriebenen oder gelesenen Daten sind bekannt. Der nächste Schritt ist die Zuweisung der Adressen an die Geräte und deren Funktion. Diese Information befindet sich bei beiden analysierten Demonstratoren in den Konfigurationsdateien der Steuerungs-Software.

Die folgenden Abschnitte behandeln die Extraktion dieser Information, und deren anschließende Verarbeitung, die letztendlich eine Aussage liefert, ob das anhand des Netzwerkverkehrs observierte Verhalten der Anlage dem gewünschten Sollzustand entspricht.

2.3.1 Interpretation von Feldbusprotokollen

Folgende Verfahren erlauben das automatisierte Zuordnen von Adressen zu Variablen im Quellcode der jeweiligen Steuerungs-Programme.

2.3.1.1 Interpretation von Modbus-Nachrichten aus OpenPLC-Steuerungen

Für die Interpretation von Modbus-Nachrichten im ersten Demonstrator werden folgende Eingabedaten benötigt:

- Die Konfigurationsdatei (**openplc_modbus_config.cfg**), sie enthält Informationen über die verfügbaren Register und die IP-Adressen der angeschlossenen Komponenten:

```
device0.name = "Ultra Sonic Level Meter (41)"
device0.protocol = "TCP"
device0.slave_id = "1"
device0.address = "10.42.0.41"
device0.IP_Port = "502"
device0.Input_Registers_Start = "0"
device0.Input_Registers_Size = "3"
```

- Die Zuordnungsanweisung in den Adressraum der Programmierumgebung, sie ist Bestandteil von OpenPLC (www.openplcproject.com/scada):

Register Type	Usage	PLC Address	Modbus Address	Register Size	Value Range	Access
Coil Registers	Digital Outputs	%QX0.0 - %QX99.7	0 - 799	1 bit	0 or 1 OFF or ON	read and write
Discrete Input Registers	Digital Inputs	%IX0.0 - %IX99.7	0 - 799	1 bit	0 or 1 OFF or ON	read only
Input Registers	Analog Inputs	%IW0 - %IW99	0 - 1023	16 bits	0 to 65,535	read only
Holding Registers	Analog Outputs	%QW0 - %QW99	0 - 1023	16 bits	0 to 65,535	read and write

Die drei Register des oben gezeigten Ultraschall-Sensors werden demnach im Programm %IW0, %IW1 und %IW2 bezeichnet

- Abschließend lässt sich der Variablenname extrahieren, um die finale Bedeutung jedes Registers zu erfassen, diese Information befindet sich in der ST-Datei, deren Format in IEC 61131-3 [31] spezifiziert ist. Sie enthält das Betriebsprogramm der Anlage:

```
PROGRAM My_Program
  VAR
    ist AT %IW0 : WORD;
    soll_min AT %IW1 : WORD;
    soll_max AT %IW2 : WORD;
  END_VAR
  [...]
END_PROGRAM
```

Alle gezeigten Informationen lassen sich zu einem Block an Bedingungen zusammensetzen, die erfüllt werden müssen, um einen bestimmten Wert im Datenverkehr zu identifizieren. Um beispielsweise den Wert „ist“ zu extrahieren muss eine eingehende Nachricht folgende Bedingungen erfüllen (in Klammern jeweils die Bezeichnungen der Felder aus den P4 Beschreibungen, bei Modbus ist die Sensor/Aktor-Komponente der Server und die Steuerung der darauf zugreifende Client):

1. IP-Server-Adresse (meta_header.client_ip) = 10.42.0.41
2. IP-Nachricht vom Server stammend (meta_header.from_server)
3. IP-Server-Port (meta_header.server_port) = 502
4. Nachrichtentyp „Read Input Register Response“ (modbus_readir_response)
5. Nachricht muss Register 0 enthalten, also mindestens zwei Bytes lang sein (modbus_readir_response.byte_count >= 2)

Treffen alle diese Bedingungen zu, befindet sich an den ersten 16 Bit des Feldes `modbus_readir_response.registers` die Eingabewerte der Variable „ist“.

Damit ist die Interpretation aller Eingabe- und Ausgabewerte einer Modbus-Kommunikation möglich, der nächste Schritt ist jetzt, diese auf Einhaltung von Grenzwerten, Abweichungen von Sollwerten oder Plausibilität zu prüfen.

2.3.1.2 Interpretation von Profinet-Datenpaketen in Simatic-Installationen

Im Gegensatz zu den festen Zuordnungen der Werte in den Datenfeldern der Modbus-Kommunikation werden die Positionen der Werte in Profibus-Netzen beim Start der Steuerung dynamisch zugewiesen. Dazu dient das Profinet Context Manager Protokoll, das auf DCE/RPC und UDP aufbaut. Eine mögliche Herangehensweise ist das Implementieren eines P4-Parsers zu Extraktion der Wertzuordnungen aus dem Context Manager Protokoll.

Die Informationen die übertragen werden stammen aus der Konfigurationsdatei der Steuerung. Im untersuchten Demonstrator ließ es sich bewerkstelligen, die Wertzuordnungen direkt aus dieser Datei abzuleiten, da die Werte in identischer Reihenfolge per Context Manager zugeordnet werden. Dabei ist zu beachten, dass ungenutzte Bytes oder unbenannte Adressen (oberhalb von 8000) nicht zugeordnet werden.

Mit einem für diesen Zweck entwickelten Python-Skript kann ein P4-Parser aus der Konfigurationsdatei generiert werden.

Ausschnitt aus Eingabedatei `simatic300.cfg`

```
IOSUBSYSTEM 100, IOADDRESS 1, SLOT 36, "8DI DC24V"
  ADDRESS 16, 0, 8, 0, 0, 16
  SYMBOL I , 5, "lSchranke0fen", ""
IOSUBSYSTEM 100, IOADDRESS 1, SLOT 37, "8DO DC24V/0,5A"
  ADDRESS 16, 0, 8, 0, 0, 16
  SYMBOL O , 0, "mOfenSchieberAus", ""
  SYMBOL O , 2, "mOfenSchieberEin", ""
IOSUBSYSTEM 100, IOADDRESS 1, SLOT 38, "2AI U ST"
  ADDRESS 256, 0, 4, 0, 0, 32
  SYMBOL I , 0, "farbe1Kanal", ""
IOSUBSYSTEM 100, IOADDRESS 1, SLOT 39, "2AI I 2WIRE ST"
  ADDRESS 260, 0, 4, 0, 0, 32
IOSUBSYSTEM 100, IOADDRESS 1, SLOT 40, "2AI I 2/4WIRE HF"
  ADDRESS 264, 0, 4, 0, 0, 32
```

Ausschnitt aus den daraus generierten P4-Quellen:

```
header_type pnober_input_t {
  fields {
    [...]
    lSchranke0fenpadding : 2;
    lSchranke0fen : 1;
  }
}
```



```
padding15 : 5;
slot36IOPS : 8;
slot37IOPS : 8;
farbe1Kanal : 32;
farbe1KanalIOPS : 8;
slot39data : 32;
slot39IOPS : 8;
slot40data : 32;
slot40IOPS : 8;
    }
}
```

Der generierte Parser agiert als zweite Verarbeitungsstufe nach dem primären Parser, der den Ethernet- und Profinet-Header verarbeitet. So lassen sich die Werte mit einem reduzierten Satz an Bedingungen extrahieren: Es muss nur im IOPS-Feld geprüft werden, ob die transportierten Daten valide sind.

2.3.2 Implementierung von Interpretern

Um die aus den Datenpaketen extrahierten Werte nutzen zu können, wird eine Methode benötigt, mit der sich korrektes Verhalten und/oder Fehlverhalten modellieren lässt. Ein Vergleich der Ausgabewerte aus dem Verhaltensmodell mit den aufgezeichneten Daten aus dem Netzwerkverkehr führt dann zu einer Aussage über einen möglichen Fehler oder aufgetretener Anomalie.

Da das Verhalten einer Steuerung definiert ist über das dort ausgeführte Programm ist eine mögliche Herangehensweise, dieses Programm zu simulieren und anschließend dessen Ausgaben mit der zu überwachenden Steuerung zu vergleichen.

In den Demonstratoren kommen zwei unterschiedliche Programmiersprachen zum Einsatz:

OpenPLC: Verwendet Structured Text, spezifiziert in IEC 61131 [IEC61131]

Step7: verwendet SCL, dies basiert ebenfalls auf Structured Text, spezifiziert in SIMATIC S7-SCL [32]

Zur Simulation der Steuerung müssen diese Programme ausgeführt werden. Dazu sind zwei Herangehensweisen denkbar:

Direkte Verarbeitung: Das Entwickeln eines Parsers sowie Interpreters und/oder Compilers für alle zu untersuchenden Zielsprachen. So können die Simulationen direkt mit den Entwicklungsdaten der Anlage gestartet werden.

Verarbeitung über Zwischensprache: Das Entwickeln einer neuen Zwischensprache, die alle Features der zu untersuchenden Sprachen bietet oder über Bibliotheken implementieren kann. Für diese Sprache wird dann ebenfalls Parser, Interpreter und ggf. Compiler entwickelt, sowie eine Ausführungsumgebung. Die zu untersuchenden Programme werden dann

via Übersetzungsroutinen in diese Sprache überführt. Dies erspart das Entwickeln mehrerer Verarbeitungs-Stacks und ist somit effizienter, da für jede Sprache nur Übersetzer-Tools benötigt werden.

Die Fehlererkennungsmethoden des Fraunhofer SIT bauen momentan auf der zweiten Option auf. Die nächsten Punkte behandeln das Design der implementierten Lösung.

2.3.2.1 Grammatikdefinition

Die Grammatik der implementierten Zwischensprache ist in Backus-Naur-Form (BNF) definiert. Sie ist ähnlich dem in IEC 61131 spezifiziertem Structured Text, der in erweiterter Backus-Naur-Form im Anhang der Spezifikation verfügbar ist. Dies erleichtert, wenn gewünscht, zum ersten Ansatz aus dem vorherigen Abschnitt zu konvertieren.

2.3.2.2 Parsererzeugung

Zur Erzeugung des Parsers wird mit dem BNF Converter [33] der Chalmers University of Technology and University of Gothenburg in Bison und Flex [34] Code überführt, welches daraus wiederum C++ Code erzeugt. Dieser Code bietet einen Rumpf der Objekte für alle Elemente der definierten Sprache enthalten. Diese Objekte werden per Visitor Pattern bei der Ausführung der traversiert. Dieser Rumpf ist mit den gewünschten Funktionalität zu füllen.

2.3.2.3 Beispielgrammatik

Das folgende Listing zeigt einen Ausschnitt aus der BNF-Definition der Zwischensprache:

```
comment      "//";
comment      "/*" "*/";

token Hexadecimal '0' ('x'|'X') (digit | ["abcdef"] | ["ABCDEF"]);

ERules.      Rules      ::= [Def];

terminator Def "";

EDef.        Def        ::= Cond ":" Ident "=" Cond;
EInitVar.    Def        ::= "Init" Ident ":" Cond;
ECodeDef.    Def        ::= CodeDef;

EVariable.   CodeDef    ::= Ident ":" Cond;
EIfDef.      CodeDef    ::= "if" "(" Cond ")" "{" [CodeDef] "}";
EIfElse.     CodeDef    ::= "if" "(" Cond ")" "{" [CodeDef] "}"
                        "else" "{" [CodeDef] "}";
```

Die ersten Zeilen definieren, wie Kommentare formuliert werden können, in dem Fall ähnlich der in der Programmiersprache C verwendeten. Anschließend wird eine Produktionsregel

anegelegt, die auf hexadezimale Werte in Format mit führendem „0x“ zutrifft. Die Angabe „Rules ::= [Def];“ legt das Programm als Liste von „Def“ Elementen an, die nicht terminiert werden.

Anschließend werden drei „Def“-Erzeuger angelegt. Ident ist dabei ein vordefiniertes Element für Namen in Programmen, das als String ausgewertet wird.

Die letzten drei Zeilen zeigen Beispiele wie sich Wertzuweisungsoperatoren und If-Else-Konstrukte anlegen lassen.

2.3.3 Extraktionsmethoden

Um die für die Extraktion der Werte benötigten Bedingungen zu formulieren wird ein „Edef“-Konstrukt angelegt, das wie folgt aufgebaut ist:

Bedingung : Variable = Extraktor

Damit lassen sich die zu erfüllenden Bedingungen zur Datenextraktion definieren, beispielweise für den Fraunhofer SIT Demonstrator:

```
(u32("meta_header.server_ip") = ip("10.42.0.41")) and
(u8("modbus_readir_response.function_code") = 4) : ist =
u16("modbus_readir_response.registers"[0])
```

Für den AP4-Demonstrator sind die Extraktoren einfacher zu erstellen, da die Parser-Zwischenstufe die Extraktion erleichtert. Es wird nur die Bedingung benötigt, die die gelieferten Daten auf Validität prüft:

```
u8("pnoben_input.farbe1KanalIOPS") & 0x80 : farbe1Kanal =
u16("pnoben_input.farbe1Kanal")
```

Ein Generator für die Extraktionsregeln aus der Konfigurationsdatei der Step 7 Entwicklungsumgebung wurde als Zusatzmodul des Zwischenstufen-Generators in das bestehende Python-Skript implementiert.

2.3.4 Fehlererkennungsmethoden – Die Simulation

Zur Simulation der Steuerungsprogramme ist die Zwischensprache mit reduziertem Umfang verfügbar. Sie besteht aus algebraischen Operatoren, Zuweisungen und einem If-Then-Else-Konstrukt. Damit können alle untersuchten Programme mit wenig Aufwand abgebildet werden. Die Programme werden im dazugehörigen Interpreter ausgeführt falls die Extraktoren eine Änderung der Eingabewerte im Netzwerkverkehr feststellen.

Das folgende Listing zeigt einen Ausschnitt aus dem Demonstrator-Programm:

```
IF (GVAR.F_one = true) THEN
  IF (GVAR.FARBE_min_farbe >= WORD_TO_INT(farbe1Kanal)) THEN
```

```

    GVAR.FARBE_min_farbe := WORD_TO_INT(farbe1Kanal);
END_IF;

GVAR.F_flanke := FARBE_iTaster AND NOT GVAR.F_signalOld;
GVAR.F_signalOld := FARBE_iTaster;

IF (GVAR.F_flanke = true) THEN
    GVAR.FARBE_bandMove := GVAR.FARBE_bandMove + 1;
END_IF;

IF (GVAR.FARBE_bandMove > 9) AND (GVAR.FARBE_bandMove < 13) THEN
    FARBE_band := false;
    GVAR.F_CurrTime := S_PULSE(T_N0:= T11, S:=TRUE, TV:= T#3s,
        R:=FALSE, BI:=GVAR.F_BiVal,Q:=GVAR.F_actFlag);

    IF (GVAR.F_actFlag = false) THEN
        GVAR.F_CurrTime := S_PULSE(T_N0:= T11, S:=FALSE, TV:=
            T#3s, R:=TRUE, BI:=GVAR.F_BiVal,Q:=GVAR.F_actFlag);
        GVAR.F_one := false;
        GVAR.F_oneB := true;
        FARBE_band := true;
        GVAR.FARBE_bandMove := 0;
    END_IF;
END_IF;
END_IF;

```

Aufgabe des gezeigten Programms ist, ein Förderband (Ausgabewert: FARBE_band) bis zu einer bestimmten Position zu fahren, die durch das Zählen von Signalfanken (Eingabewert: FARBE_iTaster) ermittelt wird. Anschließend wird drei Sekunden gewartet, um die Farbe des Werkstückes zu messen (Eingabewert: farbe1Kanal). Das Warten ist durch einen S_PULSE Timer implementiert. Dann wird das Band wieder aktiviert um zum nächsten Bearbeitungsschritt zu gelangen.

In die Zwischensprache übersetzt zeigt sich das gleiche Programm wie folgt, hierbei wurde die S_PULSE Funktion durch die in der Zwischensprache verfügbaren Now()-Funktion emuliert, die die Zeitstempel der eintreffenden Pakete in Mikrosekunden zurückgibt:

```

if (F_one) {
    if (FARBE_min_farbe >= farbe1Kanal) {
        FARBE_min_farbe := farbe1Kanal
    }

    F_flanke := FARBE_iTaster and (not F_signalOld)
    F_signalOld := FARBE_iTaster

    if (F_flanke) {
        FARBE_bandMove := FARBE_bandMove + 1
    }

    if ((FARBE_bandMove > 9) and (FARBE_bandMove < 13)) {
        SOLL_FARBE_band := 0

        if (not T11_running) {
            T11_trigger := Now() + 3000000
        }
    }
}

```

```

        T11_running := 1
    } else {
        if (T11_trigger <= Now()) {
            T11_running := 0
            F_one := 0
            F_oneB := 1
            SOLL_FARBE_band := 1
            FARBE_bandMove := 0
        }
    }
}
}
}

```

Festzuhalten ist hier, dass alle Ausgabewerte mit einem Präfix „SOLL_“ versehen wurden. Diese Werte aus der Simulation werden später im Erkenner mit den aus dem Datenstrom extrahierten Ist-Werten verglichen.

Tools zur automatischen Übersetzung von SCL- oder ST-Code in die Zwischensprache sind denkbar und könnten im weiteren Verlauf des Projekt realisiert werden.

2.3.5 Fehlererkennungsmethoden – Der Erkenner

Den abschließenden Erkennungsschritt führt die Lösung über die folgenden Methoden durch. Deren Ausgaben werden sowohl dem Bediener angezeigt als auch in eine SQL-Datenbank überführt. Aus dieser Datenbank kann dann entweder ein Leitstand direkt Daten auslesen und anzeigen, oder dies über ein zwischengeschaltetes Aggregations- und Assoziationssystem verarbeiten lassen, um dann kontextabhängige Meldungen zu erhalten.

2.3.5.1 Sollwertvergleiche

Um Fehler zu erkennen existiert folgendes Konstrukt zur Definition von abnormalen Maschinenenzuständen:

```
Anomalies Name (Triggerliste) { Bedingungsliste }
```

Diese Art der Fehlererkennung liefert sofort eine Meldung. Am Ende der Simulationsphase wird von der Ausführungsumgebung der Zwischensprache die Triggerliste ausgewertet. Hat sich einer der Werte in dieser Liste im Vergleich zur vorherigen Auswertung verändert, wird die Bedingungsliste abgearbeitet. Findet sich dort eine Bedingung, die zutrifft (also zu einem Wert ungleich 0 ausgewertet), wird eine Meldung erzeugt.

Ein Beispiel aus dem AP4-Demonstrator ist das Auslösen einer Lichtschranke während das darunterliegende Förderband nicht aktiv ist. Dies sollte zu keiner Zeit der Fall sein. Falls es doch auftritt liegt entweder ein Fehler einer Komponente vor oder die Sensoren sind verschmutzt oder ein unerlaubter Eingriff in die Maschine liegt vor. In allen Fällen soll der Leitstand sofort benachrichtigt werden, die Regel lautet dann:

```
Anomalies LichtschränkeFehler (FARBE_lSchränkeEingang l) {
    (not l) and (not FARBE_band) and (not mBandVor)
}
```

Diese Art von Fehlerdefinition eignet sich nicht für die Überwachung von Sollwerten aus der Simulation, da diese üblicherweise zu einem Soll-Wert gelangt, bevor das dazugehörige Datenpaket ausgewertet ist, welches den Ist-Wert enthält. Daher darf eine Meldung erst generiert werden, wenn ein gewisses zeitliches Limit erreicht ist. In echtzeitfähigen Systemen ist dieser Grenzwert fest definiert und daher kann die Fehlererkennung mit solchen System optimal arbeiten. Zum Anlegen zeitlich limitierter Regeln ist folgende Syntax zu verwenden:

```
Anomalies Name (Triggerliste) for Limit ms { Bedingungsliste }
```

Um diese Regeln für den vorher gezeigten Simulationscode zu nutzen, benötigt man die maximal erlaubte Reaktionszeit der verwendeten Steuerung. Diese liegt bei der untersuchten Simatic Steuerung in der Standardeinstellung bei 150 Millisekunden. Daher lautet die Erkennungsregel für den Ausgabewert des Förderbandes wie folgt:

```
Anomalies FARBE_band_Abweichung (SOLL_FARBE_band sfb, FARBE_band fb)
for 150 ms {
    sfb != fb
}
```

Die genutzten Zeitstempel zur Erkennung einer überhöhten Verzögerung werden beim Empfang der Pakete angelegt und nicht bei der Ausführung des Codes. Die Analyse kann also verzögert stattfinden, oder beliebig oft mit gleichem Ergebnis aus einer Aufzeichnung wiederholt werden, da die Aufzeichnungsdaten (wie beispielsweise solche im PCAP-Format) exakte Zeitstempel enthalten.

2.3.5.2 Zustandsübergangsüberwachung

Zur Überwachung von Zustandsänderungen im Steuerungsnetz ist zusätzlich die „Transitions“-Anweisung implementiert in der erlaubte Zustandswechsel aufgelistet werden. Findet in der Anlage ein Wechsel eines Ausgabewertes statt, der nicht in der Liste enthalten ist, wird eine Fehlermeldung erzeugt. Eine Zustandsübergangsregel ist wie folgt aufgebaut:

```
Transitions Name (Triggerliste) { BedingungAlt -> BedingungNeu, ... }
```

Derartige Regeln werden ausgewertet, wenn sich eine in der Triggerliste aufgeführte Variable ändert. Die Regel erzeugt keine Meldung wenn sowohl vor wie nach der Änderung

die „BedingungAlt“ zutrifft, oder wenn vor der Änderung „BedingungAlt“ und danach „BedingungNeu“ zutrifft. In allen anderen Fällen wird eine Warnmeldung generiert.

Im AP4-Demonstrator nutzt das Sortierwerk einen Kompressor und drei Ventile um Werkstücke in Sortierkörbe zu schieben. Die korrekte Abfolge des Prozesses ist:

- Der Sortierer ist nicht aktiv (Kompressor aus, Ventile geschlossen)
- Der Kompressor wird aktiviert, die Ventile sind noch geschlossen
- Nur ein Ventil öffnet sich und schließt wieder um das Werkstück zu schieben
- Der Kompressor schaltet wieder ab, der Prozess kann erneut beginnen

Dieser Vorgang ist fest programmiert und erlaubt keine Abweichungen. Daher deuten alle anderen Vorgänge auf dem Kompressor oder den Ventilen auf eine kompromittierte Steuerung hin. Mit einer „Transitions“-Anweisung kann das Verhalten überwacht werden:

```
Transitions KompressorVentilAbfolge (FARBE_kompressor komp,
FARBE_ventilW w, FARBE_ventilR r, FARBE_ventilB b) {

    //Kompressor geht nur an während alle Ventile geschlossen sind
    (not w) and (not r) and (not b) and (not komp)
    -> (not w) and (not r) and (not b) and komp,

    //Anschließend öffnet sich genau eines der Ventile
    (not w) and (not r) and (not b) and komp
    -> w and (not r) and (not b) and komp,
    (not w) and (not r) and (not b) and komp
    -> (not w) and r and (not b) and komp,
    (not w) and (not r) and (not b) and komp
    -> (not w) and (not r) and b and komp

    //Das offene Ventil schließt und der Kompressor schaltet ab
    w and (not r) and (not b) and komp
    -> (not w) and (not r) and (not b) and (not komp),
    (not w) and r and (not b) and komp
    -> (not w) and (not r) and (not b) and (not komp),
    (not w) and (not r) and b and komp
    -> (not w) and (not r) and (not b) and (not komp)
}
```

2.3.5.3 Beispielausgaben

Die folgende Liste zeigt beispielhaft die Ausgaben der implementierten Erkennungsroutinen. Zusätzliche Informationen zur Variablenbelegung werden in der Datenbank hinterlegt.

```
FAIL: Packet      n/a: Timeout: Kompressor: Abnormal state persisted
OK   : Packet 110877: Timeout: Kompressor: Recovered after 2193.930 ms
OK   : Packet 188093: Latency: Kompressor: 0.963 of 150 ms - 0.642%
```

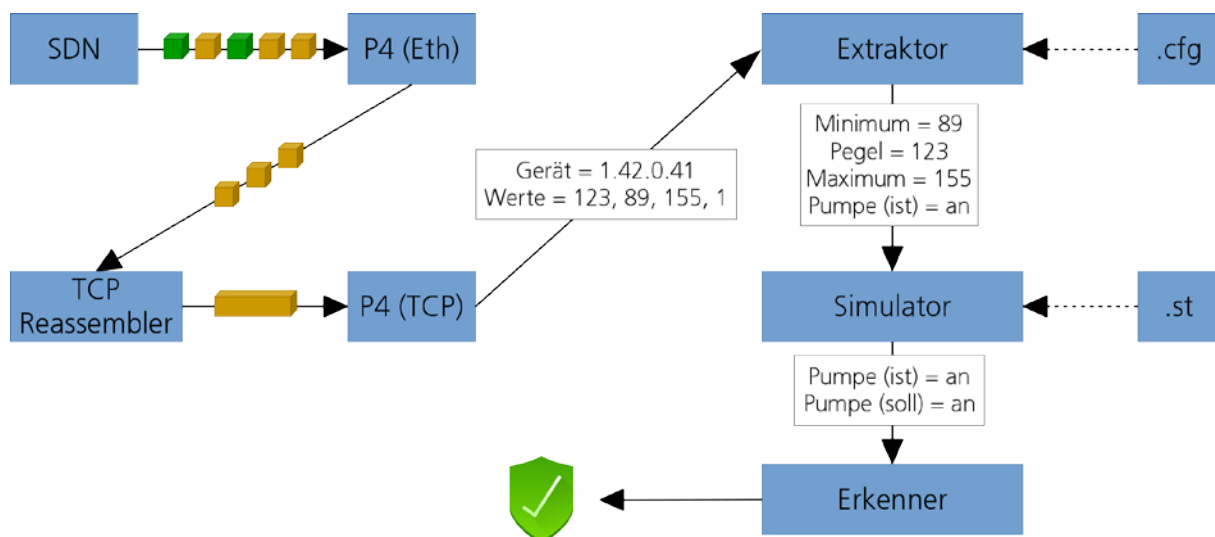
FAIL: Packet 167142: Anomaly: FARBE_band: VIOLATION DETECTED

OK : Packet 182715: Transit: KVAbfolge: Clean transition

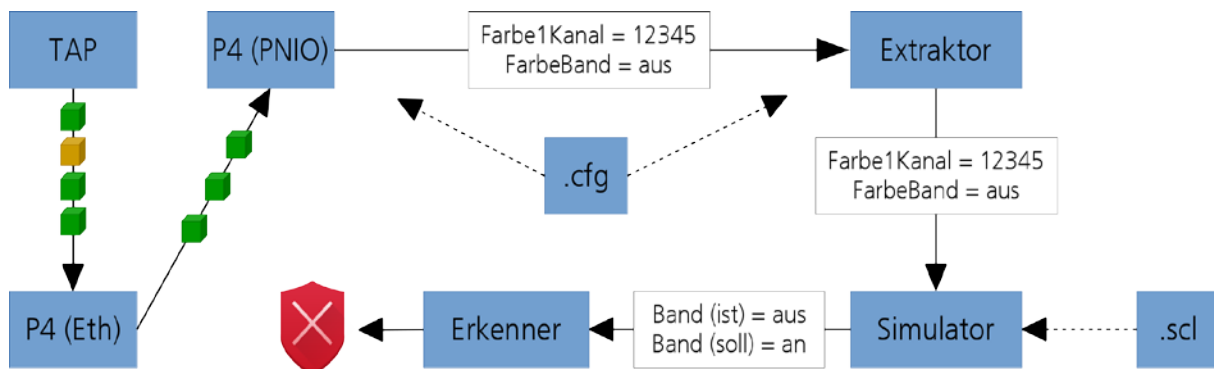
FAIL: Packet 188093: Transit: KVAbfolge: INVALID TRANSITION

Die ersten drei Zeilen zeigen die Ausgabe einer „Anomalies“-Regel mit Zeitlimit. Die erste Meldung wird beim Überschreiten des Limits erzeugt, die zweite wenn der Fehlerzustand wieder aufgelöst wird. Die dritte Zeile zeigt den Fall wenn die Regel rechtzeitig erfüllt wird. In diesem Falle wird die Reaktionszeit der SPS angezeigt. Die vierte Zeile zeigt die Meldung einer nicht zeitgebundenen „Anomalies“-Regel, die letzten beiden Zeilen eine erfolgreiche und eine ungültige Auswertung einer „Transitions“-Anweisung.

2.3.6 Gesamtarchitektur



Diese Grafik zeigt die Gesamtansicht der Lösung im Kontext des Fraunhofer SIT Demonstrators. Die vom SDN-Switch extrahierten Paketen werden von der ersten P4-Parserstufe verarbeitet, anschließend werden alle TCP-Pakete an den Reassembler weitergeleitet. Dieser setzt die TCP-Ströme wieder zusammen und gibt einzelne vollständige Nachrichten an die zweite P4-Parserstufe. Diese extrahiert die Werte aus den Nachrichten. Der aus der Konfigurationsdatei abgeleitete Extraktor weist den Werten Variablen zu und gibt diese an die Simulation weiter, die aus den Werten die zu erwartenden Sollwerte errechnet. Anschließend werden die Sollwerte mit den Istwerten aus den extrahierten Werten verglichen um ein Fehlverhalten zu detektieren.



Hier ist die Architektur der Erkennungssoftware im Einsatz auf dem AP4-Demonstrator zu sehen. Die Datenpakete des Taps werden in der ersten Parserstufe verarbeitet, anschließend an die aus der Konfigurationsdatei der Step 7 Entwicklungsumgebung generierte zweite Parserstufe weitergeleitet. Dort werden Variablennamen zugewiesen, anschließend prüft der ebenfalls generierte Extraktor, ob die Anlage sich im laufenden Zustand befindet und leitet in diesem Falle die Daten weiter an den Simulator. Dieser führt das SCL-Programm aus, um die Sollwerte zu erhalten, die anschließend vom Erkennen verglichen werden.

Die gezeigten Beispieldaten stammen aus der „Sortieranlage“ des Demonstrators. Dort wird die Farbe eines Werkstückes ermittelt, das über ein Förderband läuft. Im gezeigten Fall hat ein Angreifer das Band gestoppt, obwohl sich darauf noch ein unsortiertes Werkstück befindet, was sich am gemessenen Farbwert zeigt. Die Simulation erwartet, dass die Sortierung durchgeführt wird und setzt den Sollwert auf „an“. Diese Diskrepanz wird anschließend an den Leitstand gemeldet.

Hinweis: Es handelt sich bei beiden Grafiken um dieselbe universell einsetzbare Implementation. Beide Grafiken zeigen aus Gründen der Übersichtlichkeit und Nachvollziehbarkeit nur die jeweils genutzten Elemente. Im Einsatz können beliebig viele Erkennungsroutinen gleichzeitig ausgeführt werden, unabhängig von den genutzten Protokollen. Dazu muss nur sichergestellt werden, dass keine Namenskollisionen auftreten, etwa durch Nutzung von Namenspräfixen.

2.3.7 Perspektiven

Folgende Erweiterung und Optimierungen können entwickelt werden, um die Lösung zu verbessern:

Effizienz: Eine effizientere Programmierung und den Einsatz von Compilern statt Interpretern kann die Ausführungsgeschwindigkeit deutlich erhöhen, sodass die Überwachung mehrerer Maschinen ermöglicht wird. Hierzu könnte aus der Zwischensprache

C-Code erzeugt werden, der anschließend kompiliert, geladen und ausgeführt wird. Weiterhin können die auf P4 basierenden Bestandteile auf unterstützter Hardware ausgeführt werden, um die Last auf dem Auswertungsrechner weiter zu reduzieren.

Debugging: Während der Entwicklung wurde eine weitere Einsatzmöglichkeit der Methode deutlich: Der Einsatz als Werkzeug zur Analyse und zum Debugging von Steuerungen und deren Programmierung. Dies wird ermöglicht durch die Fähigkeit den Zustand der Steuerung im Detail nachzuvollziehen inklusive der Werte der Programmvariablen. Zusätzlich ist die Verwendung von Aufzeichnungen eine einfache Methode zu untersuchende Ereignisse beliebig oft zu wiederholen.

2.4 SpotFactory und N-Grams Anomaly

Name des Tools:	SpotFactory
Beschreibung:	Anomalieerkennung nach Latent Dirichlet Allocation. Als Basis wird Apache Spot verwendet, welches wir gedenken um Industrieprotokolle zu ergänzen.
Benötigter Input:	Netzwerk- und/oder Feldbustraffc
Generierter Output:	Neben den Metadaten, die beim Algorithmus anfallen, wie z.B. erkannte Topics, Wahrscheinlichkeiten von Features, ist der zentrale Output eine Warnung über eine potentielle Anomalie.
Benötigte Hard-, Software:	Apache Spot mit entsprechenden Abhängigkeiten

Name des Tools:	N-Grams Anomaly
Beschreibung:	Anomalieerkennung host- oder netzwerkbasierend auf Basis von N-Grams. bei der Netzvariante wird Netzwerktraffc überprüft wie beim oberen Tool, nur eben mit einem anderen Algorithmus. Die beiden Algorithmen werden zum einen gebenchmarkt oder aber gegenfalls als sich ergänzenden Komponenten ausgewertet. Bei der hostbasierten Variante läuft das Tool entweder auf einer Maschine, oder bezieht lokale Daten einer Maschine zur Analyse, wie z.B. Log-Daten, Systemcalls, etc.
Benötigter Input:	Netzwerk- und/oder Feldbustraffc, maschinenlokale Daten
Generierter Output:	Neben den Metadaten, die beim Algorithmus anfallen, wie z.B. erkannte Topics, Wahrscheinlichkeiten von Features, ist der zentrale Output eine Warnung über eine potentielle Anomalie.
Benötigte Hard-, Software:	In einem ersten Versuch wollen wir versuchen eine N-Gram-Library in Apache Spot einzubetten. Sollte dies nicht wie gewünscht funktionieren werden wir ein separates Werkzeug auf Basis einer noch nicht bestimmten Library (Kandidaten sind vorerst SALAD und eine SIT-interne Entwicklung)

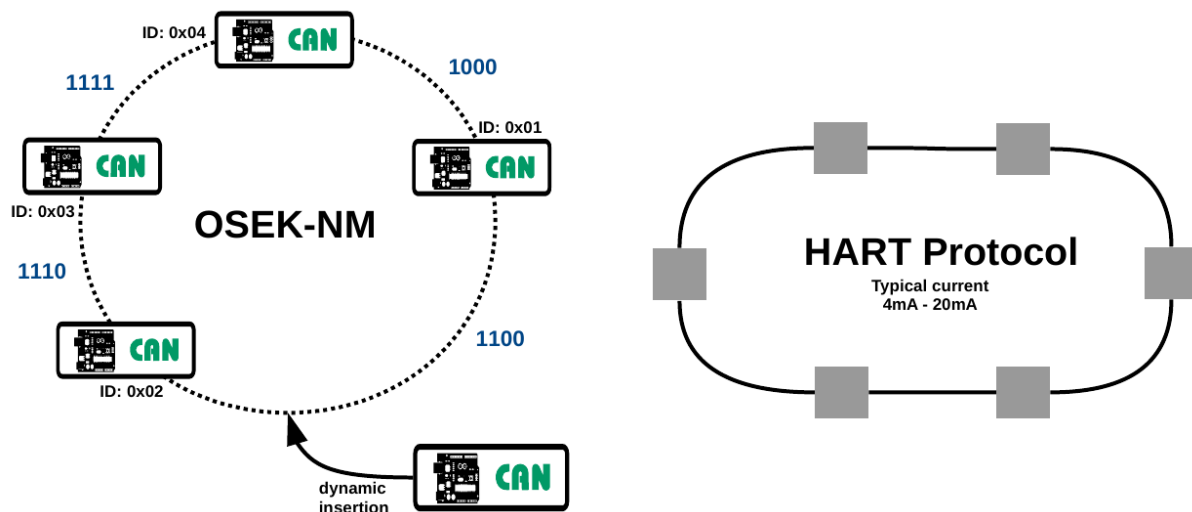
2.5 Verteiltes DFKI-Anomalieerkennungswerkzeug

Name des Tools:	Verteiltes DFKI-Anomalieerkennungswerkzeug
Beschreibung:	Verteilte Instanzen der Hardware inkl. Software werden im Netzwerk platziert. Netzwerk- und Host-basierte Daten werden gesammelt, vorverarbeitet und bewertet. Anschließend werden die gesammelten Daten an eine zentrale Stelle gesendet und im Vergleich mit anderen lokalen Instanzen bewertet. Der Fokus liegt einerseits bei der Zusammenführung von Netzwerk- und Verkehrsdaten und andererseits auf der horizontalen Aggregation, um sowohl Teil- als auch Gesamtverhalten analysieren zu können.
Benötigter Input:	- Netzwerkverkehr - Host-basierte Informationen, wie Logs und Syscalls
Generierter Output:	- Anomalielevel der überwachten Verbindung - Aggregierte Information für übergeordnete Instanz
Benötigte Hard-, Software:	- Für den Prototypen: Raspberry Pis - libpcap - C++-Compiler-Toolchain - Bibliothek zur mathematischen Analyse, wird noch bestimmt

Name des Tools:	Linux-Kernel/x86/STM32-Familie + Logboat Library
Beschreibung:	Abbildung von für die Industrie typischen Kommunikationsmustern (Analogie) auf ein CAN Netzwerk. CAN Nodes sind dabei eine Mischung aus virtualisierten und realen Nodes. Erzeugung von Anomalien, die dann erkannt werden.
Benötigter Input:	Kommunikationsmuster von Industrieellen Bussystemen für die Analogiebildung
Generierter Output:	Bereitstellung von erkannten Anomalien für die betrachteten Bus-Segemnete über das einheitliche Datenformat
Benötigte Hard-, Software:	

The diagram illustrates the IUNO architecture. At the top left, a box labeled "IUNO Leitstand" (IUNO Control Station) is connected to three "Aggregationsmodul" (Aggregation Module) blocks, each marked with a summation symbol (Σ). The top aggregation module receives data from a "Datenbasis" (Database) containing "Schichtpläne, Wartungspläne, etc." (Shift schedules, maintenance plans, etc.). The middle aggregation module receives data from four "S7 Modul" (S7 Module) blocks. The bottom aggregation module receives data from a "Virtuelle CAN Knoten" (Virtual CAN Node) cloud, which is connected to a laptop and a "USB zu CAN" interface. The bottom aggregation module also receives data from a "Laufzeit-messung" (Runtime measurement) block, which is connected to a "CAN" node. The "CAN" nodes are represented by icons of a car and a truck, with the word "CAN" in green. The "Laufzeit-messung" block is connected to a "CAN" node and a "USB zu CAN" interface. The "USB zu CAN" interface is connected to a laptop. The "Virtuelle CAN Knoten" cloud contains four "CAN" nodes. The "Laufzeit-messung" block is connected to a "CAN" node and a "USB zu CAN" interface. The "CAN" nodes are represented by icons of a car and a truck, with the word "CAN" in green. The "Laufzeit-messung" block is connected to a "CAN" node and a "USB zu CAN" interface. The "CAN" nodes are represented by icons of a car and a truck, with the word "CAN" in green.

Wie in der obigen Grafik dargestellt, besteht die momentan im Aufbau befindliche Anordnung aus realen Feldbusknoten, in welche zusätzlich auch virtualisierte Strukturen integrierbar sind. Durch die Kombination von realen und virtuellen Knoten ist der Aufbau großer Infrastrukturen mit geringem Aufwand möglich. Gleichzeitig kann aber auch ein geeignetes Demonstrations-Szenario bei Bedarf sehr kompakt umgesetzt werden.



Um möglichst realistische Kommunikationspakete zu generieren, werden aktuell aus verschiedenen industriellen Kommunikationsstandards gemeinsame Muster identifiziert. Diese werden dann auf die CAN Testumgebung adaptiert. Ein Beispiel für ein Kommunikationsmuster, bei welchem eine Verfügbarkeitsprüfung realisiert wird, ist in obiger Abbildung veranschaulicht. Im HART Protokoll ist diese als Stromschleife realisiert, welche eine Detektion von Unterbrechungen ermöglicht. Eine Entsprechung für dieses Muster findet sich bei CAN im Protokoll OSEK-NM. Dieses Protokoll setzt die Verfügbarkeitsprüfung als logischen Kommunikationsring um.

Abweichungen von etablierten Mustern, werden in diesem Versuchsaufbau als Anomalien eingestuft und durch noch zu erarbeitete Maßnahmen detektierbar gemacht.

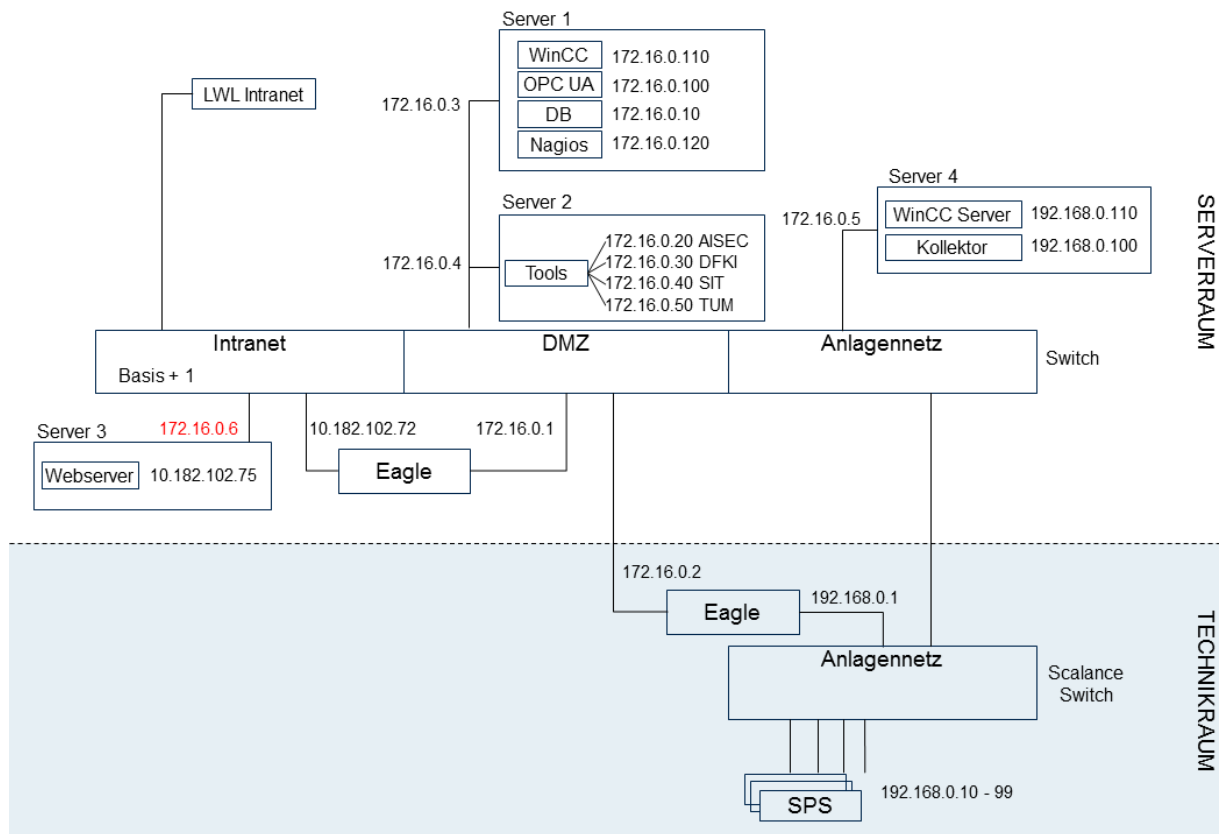
3 Aufbau der Datenaustauschplattform

Um eine valide Testumgebung für die einzelnen Anomaliedetektionswerkzeuge und das Aggregationssystem zur Verfügung zu stellen, wird im VW Werk Wolfsburg eine Infrastruktur aus Hard- und Softwarekomponenten aufgebaut. Hierzu wird auf Hardware gesetzt, wie sie auch im regulären Produktionsablauf des Werkes genutzt wird. Als Softwarelösung wird ein breites Spektrum an Programmen genutzt, um den Anforderungen des gesamten Prozesses von der Anomalieerkennung bis hin zur Bearbeitung durch einen Mitarbeiter im Leitstand zu gewährleisten.

Der gesamte Aufbau samt Verkabelung, Konfiguration und Verkabelung wird vorgenommen. Über einen industriellen Switch der Firma Hirschmann werden virtuelle Netze aufgespannt, die die Netzwerk-Infrastruktur der Datenaustauschplattform darstellen. Vier Server werden in Betrieb genommen und mit der benötigten Software ausgestattet. Der erste Server beinhaltet einen WinCC sowie OPCUA-Clients, die genutzt werden können, um diverse Testszenarien zu erzeugen, in dem bestimmter Netzwerkverkehr erzeugt wird. Des Weiteren laufen eine Instanz des Netzwerkanalysewerkzeugs Nagios sowie die Datenbank auf dem ersten Server. Letztere dient zum Sammeln und Bereitstellen der Daten des Aggregationssystems. Der zweite Server enthält die Virtualisierungssoftware, um die einzelnen Anomalieerkennungsprogramme aufnehmen und ausführen zu können. Des Weiteren wird der Server so konfiguriert, dass er das in Entwicklung befindliche Aggregationssystem bereitstellen kann. Der dritte Server dient als Webserver und wird mit gebräuchlicher Software ausgestattet, um den im Haus entwickelten visuellen Leitstand bereitstellen zu können. Der Webserver erhält eine Verbindung zum Intranet, um so als Schnittstelle zwischen aufgebauter Serverinfrastruktur und der Anwendersoftware – also dem Leitstand aus Nutzersicht – zu fungieren. Der vierte und letzte Server dient zum einen als WinCC-Server und zum anderen als Kollektor, welcher dazu dient, um die Informationen aus dem Anlagennetz zu sammeln und gebündelt und vorgefiltert an die einzelnen Anomalieerkennungswerkzeuge weiterzugeben. Die einzelnen Netze werden über manuell konfigurierte Eagle-Router der Firma Hirschmann verwaltet.

Neben den Servern wird ein Anlagennetz bestehend aus industriell eingesetzten Siemens-Komponenten aufgebaut und eingesetzt, welches zur Erzeugung von Beispielszenarien genutzt wird.

Ein vereinfachter Plan des Aufbaus ist in der beigefügten Darstellung vorzufinden. Auf Gründen der Übersichtlichkeit wurden die benötigten TAPs, welche an allen relevanten Netzschnittstellen angebracht werden, nicht in die Zeichnung eingepflegt.



Literaturverzeichnis

- [1] M. H. Bhuyan, D. K. Bhattacharyya und J. K. Kalita, „Network Anomaly Detection: Methods, Systems and Tools,“ *IEEE COMMUNICATIONS SURVEYS & TUTORIALS*, 2014.
- [2] P. Winter, H. Lampesberger, M. Zeilinger und E. Hermann, „Anomalieerkennung in Computernetzwerken,“ *DuD – Datenschutz und Datensicherheit*, pp. 235-239, 2011.
- [3] *IEC 62351: Power systems management and associated information exchange – Data and communications security (Mehrteiliger Standard mit technischen Richtlinien und technischen Reports).*
- [4] *IEC/TS 61850: Communication networks and systems in substations. (Mehrteiliger Standard mit technischen Richtlinien und technischen Reports).*
- [5] *IEC 62264: Enterprise-control system integration. (Mehrteiliger Standard)..*
- [6] E. D. Knapp und J. Langill, *Industrial Network Security: Securing Critical Infrastructure Networks for Smart Grid, SCADA, and Other Industrial Control Systems*, 2011.
- [7] M. Assante, „Seeing Through the Fog of Complexity. How a foundational tool like NexDefense’s Sophia can ensure ICS/SCADA systems stay reliable, predictable, and secure,“ NexDefense, Inc., 2014.
- [8] D. Aumüller, „Vergleich einer kommerziellen Intrusion Detection Software mit einer OpenSource-Lösung im Bezug aus Kosten, Leistung, Anwendbarkeit und Kosten. Diplomarbeit FH Worms,“ 2011.
- [9] radiflow, „iSID SCADA Intrusion Detection. Data Sheet.“
- [10] SecurityMatters, „SilentDefense ICS. Product datasheet.,“ 2015.
- [11] Tofino Security, „TofinoTM Xenon Security Appliance Data Sheet DS-TSA-XENON Version 6.0“.
- [12] J. Reed, „Israel cyber-security expertise lures growing share of investment,“ *Financial Times*, Tel Aviv, 2016.
- [13] P. Suciu, „Why Israel dominates in cyber security,“ *Fortune*, 2015.
- [14] D. Shamah, „How Israel became the go-to place for cyber-security,“ *The Times of Israel*,

2016.

- [15] T. T. team, „2010-2017 Tcpdump/Libpcap,“ The Tcpdump team, 2016. [Online]. Available: www.tcpdump.org.
- [16] T. P. L. Consortium, „www.p4.org,“ 2017. [Online].
- [17] T. R. Alves, „Open PLC,“ [Online]. Available: www.openplcproject.com.
- [18] „Modbus Application Protocol Specification V1.1b3,“ Modbus Organization, Inc, [Online]. Available: www.modbus.org/specs.php.
- [19] S. W. G. e. a. McCauley, POX. [Online].
- [20] I. E. C. (IEC), „IEC 61158: Industrial communication networks - Fieldbus specifications,“ 2014.
- [21] I. C. Society, „IEEE Std 802.3-2015“.
- [22] I. E. C. (IEC), „IEC 61784-2 Ed 1.0 - Real-Time Ethernet Profiles, SERCOS III“.
- [23] EtherCAT, „1000 EtherCAT Specification,“ Technology Group ETG, 2013.
- [24] S. Bhardwaj, „Parsing and Masterarbeit: Analysis of Industrial Protocols to Build Real-Time Anomaly Detection Systems (WORKING TITLE),“ TU Darmstadt, 2017.
- [25] V. Paxson, „Bro: A System for Detecting Network Intruders in Real-time,“ Proceedings of 7th USENIX Security Symposium, pp. 31-51. , San Antonio, Texas, 1998.
- [26] R. Sommer, J. Amann und H. Seth, „Spicy: A unified deep packet inspection framework dissecting all your data,“ Tech. rep., ICSI, 2015. TR-15-004, 2015.
- [27] J. Bangert und Z. Nickolai, „Nail: A Practical Tool for Parsing and Generating Data Formats. OSDI,“ 2014.
- [28] E. e. a. Kohler, „The Click modular router.,“ ACM Transactions on Computer Systems (TOCS) 18.3, p. 263-297, 2000.
- [29] N. e. a. Foster, „Frenetic: A network programming language. ACM Sigplan Notices. Vol. 46. No. 9. ACM, 2011,“ ACM Sigplan Notices. Vol. 46. No. 9. ACM, 2011.
- [30] N. e. a. Borisov, „Generic Application-Level Protocol Analyzer and its Language, NDSS,“ 2007.

- [31] I. E. C. (IEC)., „IEC 61131-3:2013 - Programmable controllers - Part 3: Programming languages“.
- [32] Siemens, „Manual, Simatic S7-SCL v5.3 for S7-300/400.“, 2005.
- [33] M. Forsberg und A. Ranta, „BNF Converter. Proceedings of the 2004 ACM SIGPLAN Workshop on Haskell“.
- [34] J. Levine, „Flex & Bison.“, O'Reilly Media Inc., 2009.
- [35] E. T. Group, „ETG.1000 EtherCAT Specification“, 2013.
- [36] R. e. a. Pang, „binpac: A yacc for Writing Application Protocol Parsers.“, Proceedings of the 6th ACM SIGCOMM conference on Internet measurement. ACM, 2006.